



Rapport de projet d'intégration UniSalle



Réalisé par :

Sakhri Ranim

Hayouni Mohamed Amine

Encadré par:

Mme Elleuch Nesrine

Année Universitaire : 2025-2026

Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude envers Madame Elleuch Nesrine, notre encadrante pédagogique, pour l'aide précieuse qu'elle nous a apportée tout au long de ce projet. Sa disponibilité, ses conseils avisés et ses orientations pertinentes ont été déterminants dans la réussite de ce travail. Sa rigueur scientifique et son engagement pédagogique ont constitué pour nous un exemple que nous nous efforcerons de suivre.

Nous adressons également nos sincères remerciements à l'ensemble du corps enseignant de l'Institut des Études Technologiques de Radès (ISET Radès) pour la qualité de la formation dispensée tout au long de notre cursus universitaire. Les connaissances et compétences acquises au fil des années ont constitué un socle solide sur lequel nous avons pu construire ce projet.

Nos remerciements vont aussi à nos familles et proches pour leur soutien moral indéfectible, leur patience et leurs encouragements constants qui nous ont permis de mener ce projet à son terme dans les meilleures conditions possibles.

Enfin, nous remercions tous ceux qui, de près ou de loin, ont contribué à l'accomplissement de ce projet d'intégration. Chaque échange, chaque suggestion et chaque retour d'expérience ont enrichi notre réflexion et amélioré la qualité de notre travail.

Dédicaces

Je dédie ce travail à ...

À mes chers parents, dont le soutien, l'amour et les sacrifices ont rendu possible l'aboutissement de mes études. Aucune dédicace ne saurait exprimer la profondeur de mes sentiments envers vous.

À mon binôme et ami Mohammed Amine Hayouni, pour son engagement total, sa rigueur et l'excellence de notre collaboration tout au long de ce projet.

À tous mes amis et collègues de promotion, pour les moments de partage, d'entraide et d'apprentissage mutuel.

— **Ranim Sakhri**

Je dédie ce travail à ...

À mes parents bien-aimés, piliers de ma vie, qui ont toujours cru en moi et m'ont soutenu sans réserve dans chacune de mes entreprises.

À ma binôme Ranim Sakhri, pour sa sérieuse implication, sa créativité et la qualité du travail d'équipe que nous avons accompli ensemble.

À tous ceux qui ont marqué mon parcours universitaire et contribué à ma formation intellectuelle et humaine.

— **Mohammed Amine Hayouni**

Résumé

Le présent rapport décrit le développement de la plateforme web UniSalles, une application dédiée à la gestion centralisée des salles universitaires au sein d'un établissement d'enseignement supérieur. Ce projet d'intégration, réalisé dans le cadre de l'année universitaire 2025–2026, répond à une problématique concrète rencontrée dans les instituts : la gestion manuelle des réservations de salles, source de conflits, d'erreurs et d'inefficacité organisationnelle.

UniSalles propose une solution complète et moderne articulée autour de trois profils d'utilisateurs distincts : le chef de département, les enseignants et les étudiants. Le chef de département dispose d'un tableau de bord centralisé pour gérer les salles, les classes, les séances et traiter les réclamations soumises par les enseignants. Ces derniers peuvent, quant à eux, consulter les disponibilités des salles, effectuer des réservations et transmettre des réclamations. Les étudiants, enfin, accèdent à leur emploi du temps et reçoivent des notifications relatives aux changements de séances.

L'application a été développée en suivant l'architecture MVC (Modèle-Vue-Contrôleur) avec le framework Laravel (PHP) pour le back-end, le moteur de templates Blade pour les vues, et Bootstrap couplé à JavaScript pour le front-end. La base de données relationnelle MySQL assure la persistance des données, tandis que le système d'authentification Laravel garantit la sécurité et la gestion des rôles.

Ce rapport présente successivement le contexte et la problématique du projet, l'analyse des besoins, la conception par diagrammes UML, les choix techniques effectués, la description des fonctionnalités implémentées ainsi que les tests réalisés.

Table des matières

Contents

Remerciements	1
Dédicaces	2
Résumé.....	3
Table des matières	4
Liste des figures	7
Introduction générale	8
Chapitre 1 : Présentation générale du projet	9
1.1 Contexte du projet.....	9
1.2 Problématique	9
1.3 Objectifs du projet	10
1.4 Étude de l'existant :	10
1.4.1 Solutions manuelles	10
1.4.2 Tableaux et outils bureautiques	11
1.4.3 Logiciels spécialisés	11
1.4.4 Synthèse critique	11
1.5 Solution proposée	11
Chapitre 2 : Analyse des besoins et conception	13
2.1 Identification des acteurs	13
2.2 Besoins fonctionnels	13
2.2.1 Module d'authentification	13
2.2.2 Module Chef de département	13
2.2.3 Module Enseignant	14
2.2.4 Module Étudiant.....	14
2.3 Besoins non fonctionnels	14
2.3.1 Sécurité	14
2.3.2 Performance.....	14
2.3.3 Fiabilité	14
2.3.4 Utilisabilité	15
2.3.5 Maintenabilité	15
2.4 Diagrammes UML	15
2.4.1 Diagramme de cas d'utilisation.....	15
.....	16

2.4.2 Diagramme de classes	16
2.4.3 Diagrammes de séquence	17
2.5 Conception de la base de données	23
Chapitre 3 : Réalisation technique	24
3.1 Architecture générale	24
3.2 Architecture MVC	24
3.2.1 Le Modèle (Model).....	24
3.2.2 La Vue (View)	24
3.2.3 Le Contrôleur (Controller).....	25
3.3 Technologies utilisées	25
3.3.1 Justification des choix technologiques	26
3.4 Structure de la base de données	26
Table users.....	26
Table enseignants	26
Table etudiants	26
Table salles	26
Table classes	27
Table seances	27
Table reclamations.....	27
Table messages	27
3.5 Sécurité et authentification.....	27
3.5.1 Système d'authentification	27
3.5.2 Gestion des rôles et des accès.....	27
3.5.3 Protection CSRF.....	28
Chapitre 4 : Présentation des interfaces et fonctionnalités.....	29
4.1 Module d'authentification	29
4.1.1 Page de connexion.....	29
4.1.2 Page d'inscription (étudiants)	30
4.2 Module Chef de département	30
4.2.1 Tableau de bord.....	30
4.2.2 Gestion des salles	31
4.2.3 Gestion des classes	32
4.2.4 Gestion des séances	33
4.2.5 Gestion des réclamations	34
4.2.6 Gestion des enseignants.....	35
4.3 Module Enseignant	35
4.3.1 Tableau de bord enseignant.....	35

4.3.2 Réserveation de salles	36
4.3.3 Envoi de réclamations	37
4.3.4 Consultation de l'emploi du temps	38
4.4 Module Étudiant	38
4.4.1 Tableau de bord étudiant	38
4.4.2 Consultation de l'emploi du temps	39
4.4.3 Notifications	39
Chapitre 5 : Tests et validation	41
5.1 Tests fonctionnels	41
5.2 Résultats obtenus	42
5.3 Difficultés rencontrées et solutions apportées.....	42
5.3.1 Gestion des conflits de réservation.....	42
5.3.2 Transactions de base de données	43
5.3.3 Gestion des requêtes AJAX	43
5.3.4 Gestion des contraintes d'intégrité	43
5.4 Perspectives d'amélioration	43
Conclusion générale	45
Bibliographie et Webographie.....	46
Ouvrages et documentation technique	46
Ressources en ligne + outils IA+lien Git :	46
Annexes.....	47
Annexe A — Extrait du code source : GestionSalleController	47
Annexe B — Extrait du code source : Vérification de conflit (Réserveation)	47
Annexe C — Diagramme de Gantt du projet	47

Liste des figures

- Figure 1 — Diagramme de cas d'utilistation UML
- Figure 2— Diagramme de classes(modèle conceptuel)
- Figure 3 — Diagramme de séquence : Authentification (Login)
- Figure 4— Diagramme de séquence : Inscription (Register)
- Figure 5 — Diagramme de séquence : Gestion des salles (Chef)
- Figure 6 — Diagramme de séquence : Gestion des classes (Chef)
- Figure 7 — Diagramme de séquence : Gestion des séances (Chef)
- Figure 8 — Diagramme de séquence : Réservations (Enseignant)
- Figure 9—Page de Connexion
- Figure 10—Page d'Inscription
- Figure 11—Dashboard Chef département
- Figure 12—Interface Gestion Salles
- Figure 13—Interface Gestion Classes
- Figure 14—Interface Gestion séances
- Figure 15—Interface Gestion des Réclamations
- Figure 16—Gestion des Enseignants
- Figure 17—Dashboard Enseignant
- Figure 18—Interface Réservation Salle
- Figure 19—Interface Envoi de Réclamations
- Figure 20—Interface Emploi du temps
- Figure 21—Dashboard Etudiant
- Figure 22—Interface Emploi du temps(etudiant)
- Figure 23—Interface Notification(etudiant)

Introduction générale

L'enseignement supérieur est un secteur en constante évolution, confronté à des enjeux organisationnels toujours plus complexes. Parmi ces défis, la gestion des espaces physiques, et notamment des salles de cours, représente un point névralgique pour le bon fonctionnement des établissements universitaires. En effet, une organisation déficiente des salles peut engendrer des conflits de planning, des perturbations pédagogiques et une insatisfaction généralisée au sein de la communauté universitaire.

Face à ce constat, l'Institut des Études Technologiques de Radès (IET Radès), comme de nombreuses institutions similaires, a exprimé le besoin de se doter d'un outil numérique centralisé permettant de rationaliser et d'automatiser la gestion de ses espaces pédagogiques. C'est dans ce contexte que s'inscrit le projet UniSalles, développé dans le cadre du module de développement web et de génie logiciel pour l'année universitaire 2025–2026.

UniSalles est une plateforme web conçue pour répondre aux besoins spécifiques de trois catégories d'utilisateurs : les chefs de département, les enseignants et les étudiants. Elle offre une interface intuitive permettant la gestion des réservations de salles, le suivi des emplois du temps, la communication entre les différents acteurs et le traitement des réclamations.

Ce rapport de projet d'intégration a pour objectif de documenter l'ensemble du cycle de vie du développement d'UniSalles, depuis l'analyse des besoins jusqu'à la mise en œuvre technique, en passant par la conception architecturale et la validation fonctionnelle. Il est structuré en cinq chapitres complémentaires.

Le premier chapitre présente le contexte général du projet, la problématique identifiée, les objectifs poursuivis et la solution proposée. Le deuxième chapitre est consacré à l'analyse des besoins et à la conception via les diagrammes UML. Le troisième chapitre détaille les choix techniques opérés et l'architecture de l'application. Le quatrième chapitre décrit les interfaces et les fonctionnalités de la plateforme. Enfin, le cinquième chapitre expose les tests réalisés, les difficultés rencontrées et les perspectives d'amélioration envisagées.

Chapitre 1 : Présentation générale du projet

Ce premier chapitre a pour vocation de poser le cadre général du projet UniSalles. Après avoir présenté le contexte institutionnel dans lequel il s'inscrit, nous exposons la problématique à l'origine de sa conception, les objectifs qu'il vise à atteindre, ainsi qu'une analyse de l'existant et la solution proposée pour répondre aux besoins identifiés.

1.1 Contexte du projet

L'Institut des Études Technologiques de Radès (ISET Radès) est un établissement d'enseignement supérieur qui accueille chaque année plusieurs centaines d'étudiants répartis dans de nombreuses filières. Il dispose d'un parc de salles de cours, de laboratoires et d'amphithéâtres dont la gestion constitue un défi logistique quotidien.

Traditionnellement, cette gestion repose sur des processus manuels : tableaux d'affichage, registres papier, communications par e-mail ou de vive voix entre enseignants et administration. Ces méthodes, bien que fonctionnelles à petite échelle, montrent rapidement leurs limites dès lors que le nombre d'intervenants et de salles augmente.

Le projet UniSalles s'inscrit dans une démarche de modernisation et de digitalisation des processus administratifs et pédagogiques de l'institution. Il répond à une volonté de proposer un outil numérique centralisé, accessible à l'ensemble des acteurs concernés, permettant une gestion efficace et transparente des ressources spatiales de l'établissement.

1.2 Problématique

La gestion actuelle des salles au sein de l'institut souffre de plusieurs dysfonctionnements majeurs qui nuisent au bon déroulement des activités pédagogiques :

- **Conflits de réservation** : l'absence d'un système centralisé entraîne fréquemment des situations où deux enseignants réservent la même salle au même créneau horaire, sans qu'aucun mécanisme ne permette de détecter et de prévenir ce type de conflit.
- **Manque de visibilité** : les enseignants n'ont pas accès en temps réel à la liste des salles disponibles et doivent souvent recourir à des démarches informelles pour obtenir cette information.

- Absences non signalées : lorsqu'un cours est annulé ou reporté, l'information ne parvient pas toujours aux étudiants en temps utile, générant des déplacements inutiles et une désorganisation des emplois du temps.
- Traitement lent des réclamations : les problèmes signalés par les enseignants (matériel défaillant, salle inadaptée, etc.) suivent un circuit de traitement informel et peu traçable.
- Absence d'un canal de communication unifié : les échanges entre enseignants, étudiants et administration s'effectuent par des canaux disparates et non centralisés.

Ces dysfonctionnements ont un impact direct sur la qualité du service rendu aux étudiants et sur l'efficacité du travail administratif. Ils constituent la problématique centrale à laquelle le projet UniSalles entend apporter une réponse structurée et pérenne.

1.3 Objectifs du projet

Le projet UniSalles vise à atteindre plusieurs objectifs complémentaires, tant sur le plan fonctionnel que technique :

- Centraliser la gestion des salles en offrant une interface unique permettant au chef de département de gérer l'ensemble des ressources spatiales de l'établissement.
- Faciliter la réservation des salles pour les enseignants grâce à un système de vérification des disponibilités en temps réel et de prévention des conflits.
- Améliorer la communication entre les différents acteurs de l'institution, notamment par l'envoi de notifications aux étudiants en cas de modification de leur emploi du temps.
- Mettre en place un système de traitement des réclamations permettant aux enseignants de signaler tout problème et au chef de département d'en assurer le suivi.
- Garantir la sécurité des données et des accès grâce à un système d'authentification robuste et une gestion précise des rôles utilisateurs.
- Proposer une interface ergonomique et responsive, accessible depuis tout type d'appareil (ordinateur, tablette, smartphone).

1.4 Étude de l'existant :

Avant de concevoir UniSalles, nous avons analysé les solutions existantes dans le domaine de la gestion des salles universitaires afin d'en identifier les forces et les faiblesses.

1.4.1 Solutions manuelles

Les méthodes traditionnelles reposent sur des registres papier, des tableaux d'affichage et des échanges verbaux ou par e-mail. Si elles présentent l'avantage de la simplicité et ne nécessitent aucun investissement technologique, elles sont particulièrement sujettes aux erreurs, aux oublis et aux conflits de réservation. Elles ne permettent pas non plus une consultation en temps réel des disponibilités.

1.4.2 Tableaux et outils bureautiques

Certains établissements utilisent des tableurs (Excel, Google Sheets) pour planifier l'occupation des salles. Ces outils offrent une meilleure lisibilité que les méthodes purement manuelles, mais ils restent limités en termes de collaboration simultanée, de gestion des accès et d'automatisation des vérifications de conflits.

1.4.3 Logiciels spécialisés

Des solutions commerciales spécialisées existent sur le marché (telles que ADE Campus, Hyperplanning ou Celcat), mais elles sont généralement coûteuses, difficiles à personnaliser et inadaptées aux spécificités des établissements tunisiens. Leur déploiement nécessite de plus une infrastructure technique que tous les instituts ne possèdent pas.

1.4.4 Synthèse critique

L'analyse de l'existant révèle un manque criant d'outils accessibles, gratuits ou à faible coût, adaptés aux besoins spécifiques des institutions universitaires tunisiennes et pouvant être développés et maintenus en interne. C'est précisément pour combler ce manque qu'UniSalles a été conçu.

1.5 Solution proposée

UniSalles est une application web développée avec le framework Laravel, qui offre une plateforme centralisée et sécurisée pour la gestion des salles universitaires. Elle s'articule autour de trois espaces fonctionnels correspondant aux trois profils d'utilisateurs identifiés :

- L'espace chef de département : tableau de bord statistique, gestion CRUD des salles, des classes, des enseignants et des séances, traitement des réclamations, envoi de messages aux étudiants.
- L'espace enseignant : consultation des emplois du temps, réservation de salles, envoi de réclamations.
- L'espace étudiant : consultation de l'emploi du temps de sa classe, réception de notifications.

La solution repose sur une architecture MVC (Modèle-Vue-Contrôleur) qui garantit la séparation des préoccupations, la maintenabilité du code et la scalabilité de l'application. Elle intègre un système d'authentification sécurisé avec gestion différenciée des rôles, une vérification en temps réel des disponibilités via des requêtes AJAX, et une interface responsive adaptée à tous les types d'écrans.

En conclusion de ce premier chapitre, il apparaît clairement que le projet UniSalles répond à un besoin réel et pressant. La solution proposée est à la fois ambitieuse dans ses objectifs et réaliste dans sa mise en œuvre, en s'appuyant sur des technologies éprouvées et accessibles.

Chapitre 2 : Analyse des besoins et conception

Ce deuxième chapitre constitue le cœur de la phase de conception du projet UniSalles. Il présente successivement l'identification des besoins fonctionnels et non fonctionnels, la définition des acteurs du système, puis la modélisation par diagrammes UML, avant d'aborder la conception de la base de données.

2.1 Identification des acteurs

Le système UniSalles implique trois catégories d'acteurs principaux, chacun disposant de droits et de fonctionnalités spécifiques :

- **Le Chef de département (Administrateur)** : il est le gestionnaire principal de la plateforme. Il administre l'ensemble des ressources (salles, classes, enseignants, séances) et supervise les activités des autres utilisateurs. Il traite les réclamations et communique avec les étudiants.
- **L'Enseignant** : il utilise la plateforme pour consulter les emplois du temps, réserver des salles et soumettre des réclamations. Il peut également accéder à ses séances planifiées.
- **L'Étudiant** : il accède en lecture à l'emploi du temps de sa classe et reçoit des notifications en cas de modification ou de rattrapage.

2.2 Besoins fonctionnels

2.2.1 Module d'authentification

- Inscription des étudiants avec création automatique d'un compte utilisateur et d'un profil étudiant.
- Connexion sécurisée avec redirection différenciée selon le rôle (chef, enseignant, étudiant).
- Déconnexion sécurisée avec destruction de la session.

2.2.2 Module Chef de département

- Tableau de bord affichant des statistiques clés (nombre de salles, enseignants, réclamations en cours).
- Gestion complète des salles (création, modification, suppression, consultation avec pagination).

- Gestion complète des classes (création, modification, suppression avec gestion des contraintes d'intégrité).
- Gestion des séances : planification avec affectation d'une salle, d'une classe et d'un enseignant, avec vérification des disponibilités en temps réel via AJAX.
- Gestion des enseignants : ajout, modification et suppression de comptes enseignants.
- Traitement des réclamations soumises par les enseignants.
- Envoi de messages (notifications) aux étudiants.

2.2.3 Module Enseignant

- Consultation des séances planifiées (emploi du temps).
- Réservation de salles avec sélection de la date, du créneau horaire, de la classe et de la salle disponible.
- Vérification dynamique des créneaux disponibles selon la date sélectionnée.
- Soumission de réclamations au chef de département.
- Annulation de réservations existantes.

2.2.4 Module Étudiant

- Consultation de l'emploi du temps de sa classe.
- Réception de notifications (messages) envoyées par le chef de département.

2.3 Besoins non fonctionnels

2.3.1 Sécurité

La plateforme met en œuvre plusieurs mécanismes de sécurité : authentification par session Laravel, protection pour tous les formulaires, gestion des accès par middleware selon les rôles, et validation des données côté serveur pour toutes les entrées utilisateur.

2.3.2 Performance

Les requêtes de vérification de disponibilité sont traitées pour éviter le rechargement complet de la page. La pagination des listes limite le volume de données chargées à chaque requête. Les requêtes Eloquent sont optimisées avec l'usage de relations eager loading.

2.3.3 Fiabilité

Un système de vérification des conflits est intégré lors de toute tentative de réservation. Les suppressions sensibles (classes utilisées en tant que clés étrangères) sont protégées par la gestion des exceptions QueryException de Laravel.

2.3.4 Utilisabilité

L'interface est conçue avec Bootstrap pour garantir une expérience utilisateur responsive et cohérente sur tous les types d'écrans. La navigation est simplifiée grâce à des menus clairs et une organisation logique des fonctionnalités.

2.3.5 Maintenabilité

Le code est organisé selon le patron architectural MVC de Laravel, garantissant une séparation nette entre la logique métier, la présentation et les données. Le nommage des classes, méthodes et variables suit les conventions PSR de PHP.

2.4 Diagrammes UML

2.4.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation modélise les interactions entre les acteurs et le système. Il met en évidence les fonctionnalités accessibles à chaque profil d'utilisateur. Le chef de département dispose des droits les plus étendus, incluant la gestion de l'ensemble des entités du système. L'enseignant peut réserver des salles et soumettre des réclamations. L'étudiant dispose d'un accès en lecture à son emploi du temps.

Les principaux cas d'utilisation identifiés sont : Se connecter, Gérer les salles (CRUD), Gérer les classes (CRUD), Planifier des séances, Réserver une salle, Consulter l'emploi du temps, Envoyer une réclamation, Traiter une réclamation, et Envoyer un message.

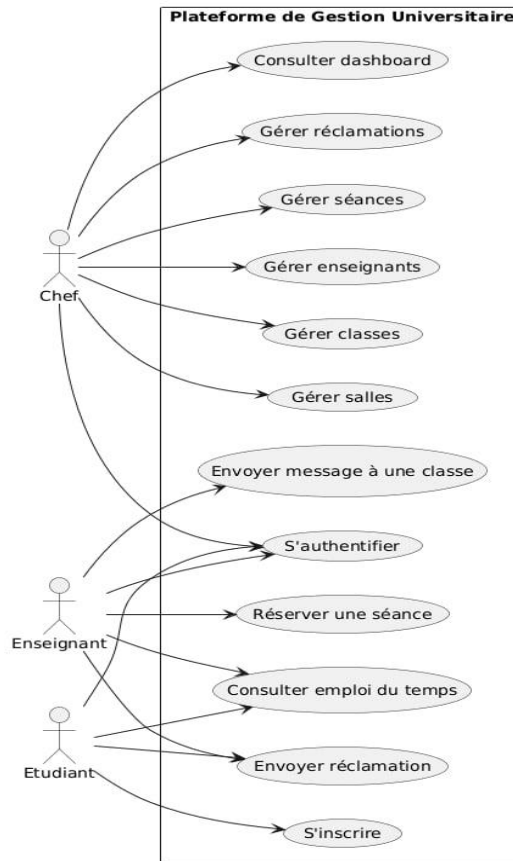


Figure 1: Diagramme de cas d'utilisation

2.4.2 Diagramme de classes

Le diagramme de classes (Figure 1) représente la structure statique du système UniSalles. Il met en évidence les entités principales du domaine et leurs relations.

La classe User est la classe centrale du système. Elle porte les attributs communs à tous les utilisateurs (identifiant, nom, e-mail, mot de passe, rôle) et est spécialisée en trois sous-classes : Chef, Enseignant et Etudiant. Cette hiérarchie reflète la relation d'héritage entre ces entités, toutes partageant un compte utilisateur de base.

La classe Salle représente une salle de cours avec ses caractéristiques (nom, capacité, type, disponibilité). La classe Classe modélise une promotion d'étudiants avec son libellé et son niveau. La classe Seance est l'entité centrale de la planification : elle lie un enseignant, une salle et une classe à un créneau horaire défini. La classe Reclamation représente une demande ou signalement soumis par un enseignant au chef de département. La classe Message permet la communication entre le chef de département et les étudiants.

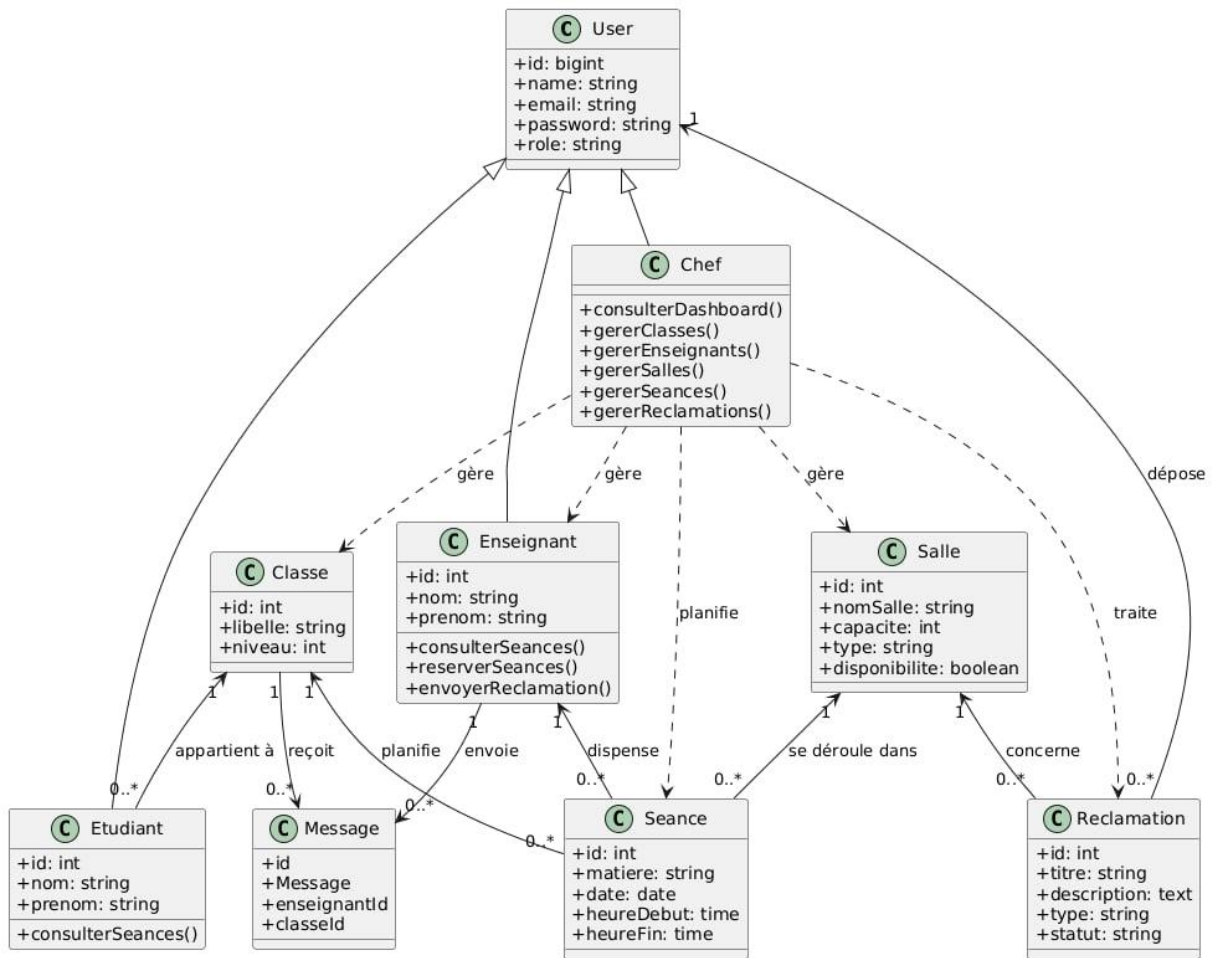


Figure 2: Le diagramme de classes

Les associations entre ces classes traduisent les règles métier du système : une séance se déroule dans une salle et concerne une classe ; un enseignant dispense des séances et peut envoyer des réclamations ; un étudiant appartient à une classe.

2.4.3 Diagrammes de séquence

Les diagrammes de séquence modélisent les interactions dynamiques entre les différents composants du système pour chaque cas d'utilisation. Nous en présentons ici les principaux.

Diagramme de séquence — Authentification (Figure 2) : Lorsqu'un utilisateur saisit ses identifiants et clique sur « Se connecter », la requête POST est transmise à l'AuthenticatedSessionController, qui délègue la validation au LoginRequest. Si les identifiants sont corrects, Laravel Auth authentifie l'utilisateur, régénère l'identifiant de session pour prévenir les attaques par fixation de session, puis redirige l'utilisateur vers le tableau de bord correspondant à son rôle (chef, enseignant ou étudiant).

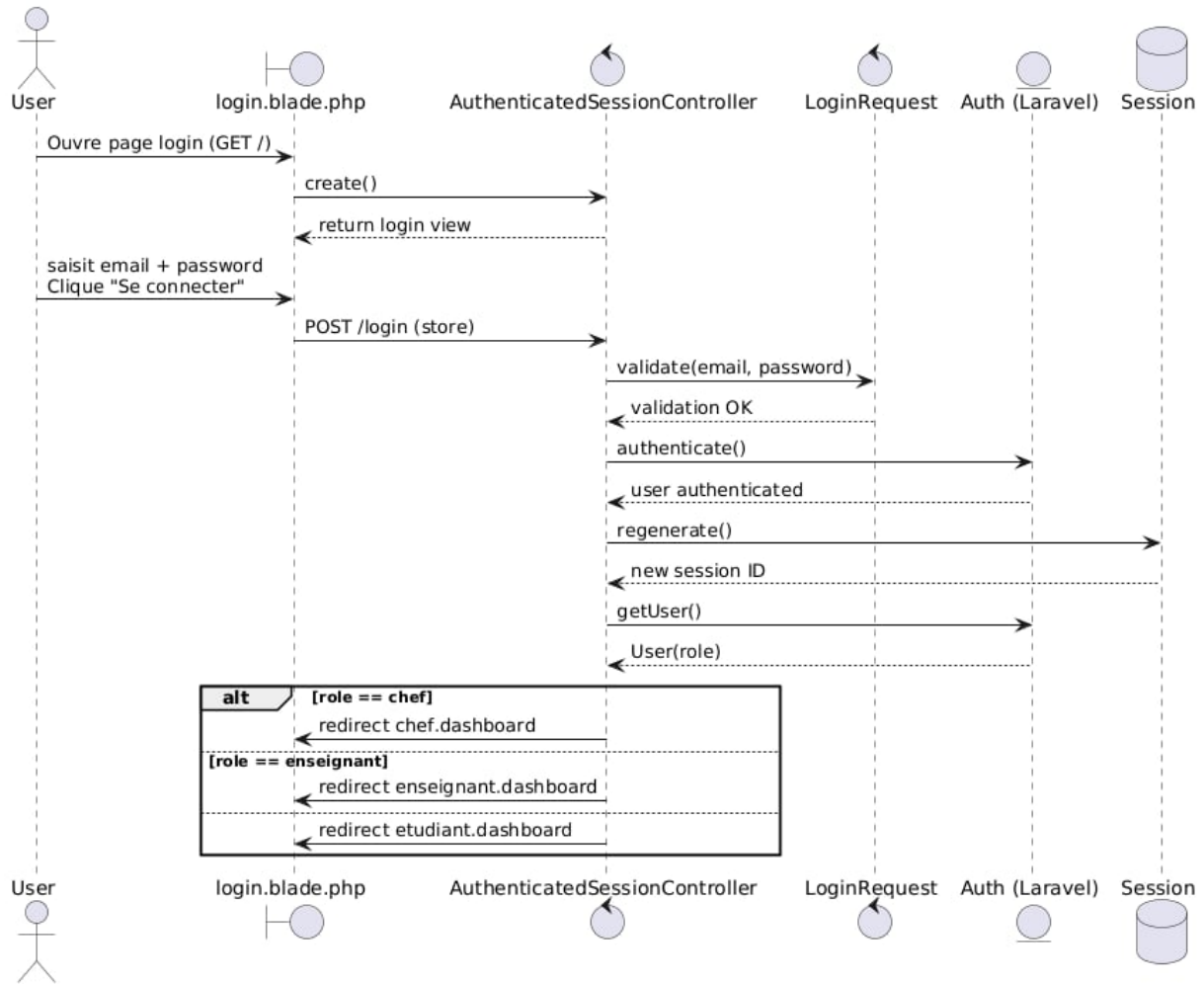


Figure 3: Diagramme de séquence — Authentification

Diagramme de séquence — Inscription (Figure 3) : Le processus d'inscription d'un étudiant implique une transaction en base de données pour garantir l'atomicité de la création simultanée du compte User et du profil Etudiant. En cas d'erreur lors de l'une de ces deux opérations, la transaction est annulée (rollback), assurant ainsi la cohérence des données.

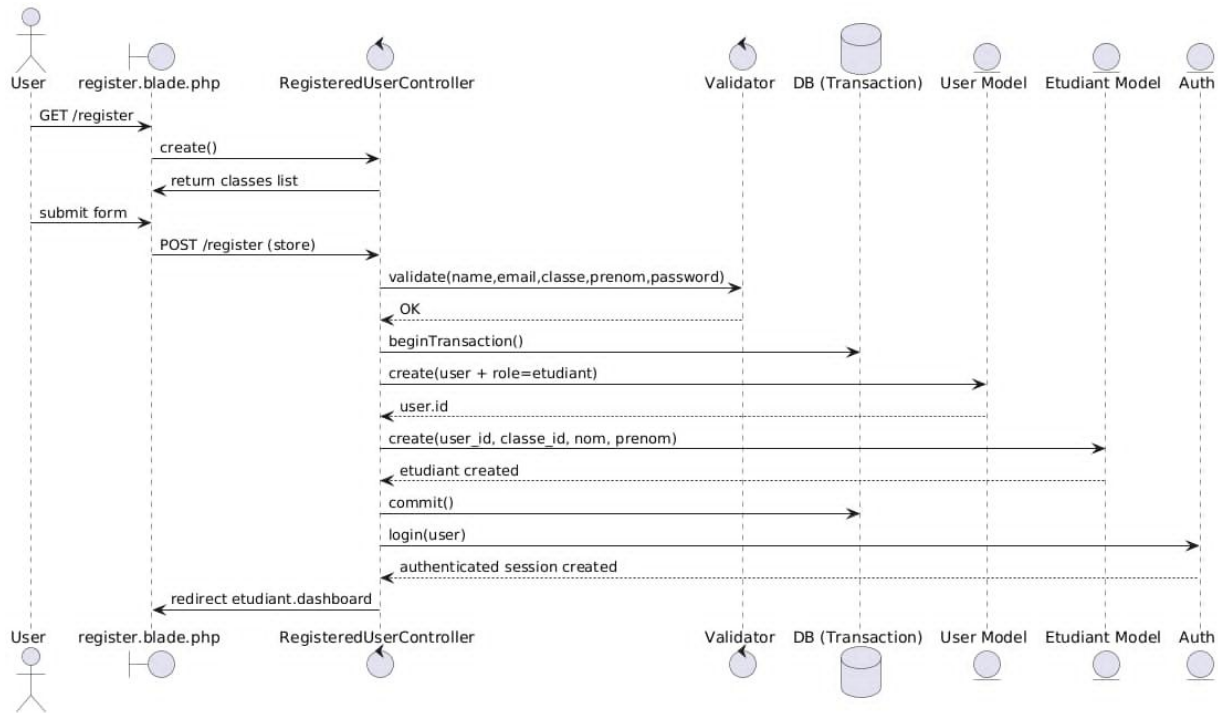


Figure 4: Diagramme de séquence — Inscription

Diagramme de séquence — Gestion des salles (Figure 4) : Ce diagramme illustre les opérations CRUD. La liste des salles est affichée avec pagination. Les opérations de création, modification et suppression suivent un flux classique de validation des données, interaction avec le modèle Eloquent, et redirection après succès.

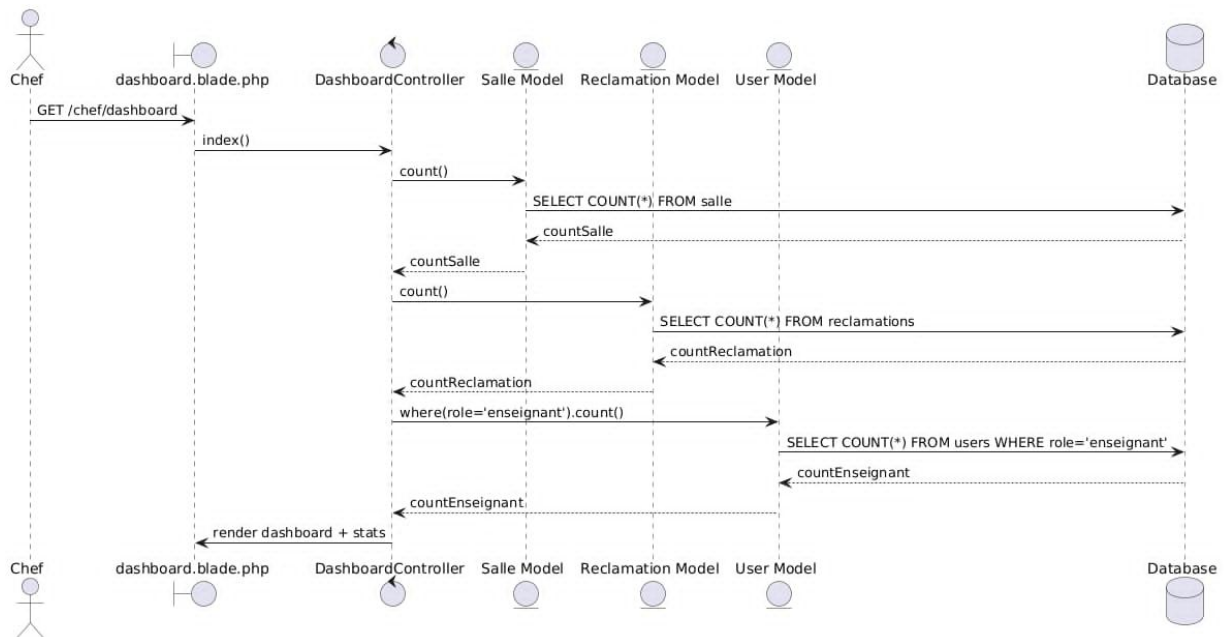


Figure 5: Diagramme de séquence — Gestion des salles

Diagramme de séquence — Gestion des classes (Figure 5) : La suppression d'une classe présente une particularité : elle est conditionnée à la vérification de l'absence de références à cette classe depuis d'autres entités (contrainte de clé étrangère). Si la classe est utilisée, une exception `QueryException` est capturée et l'utilisateur est redirigé avec un message d'erreur explicatif.

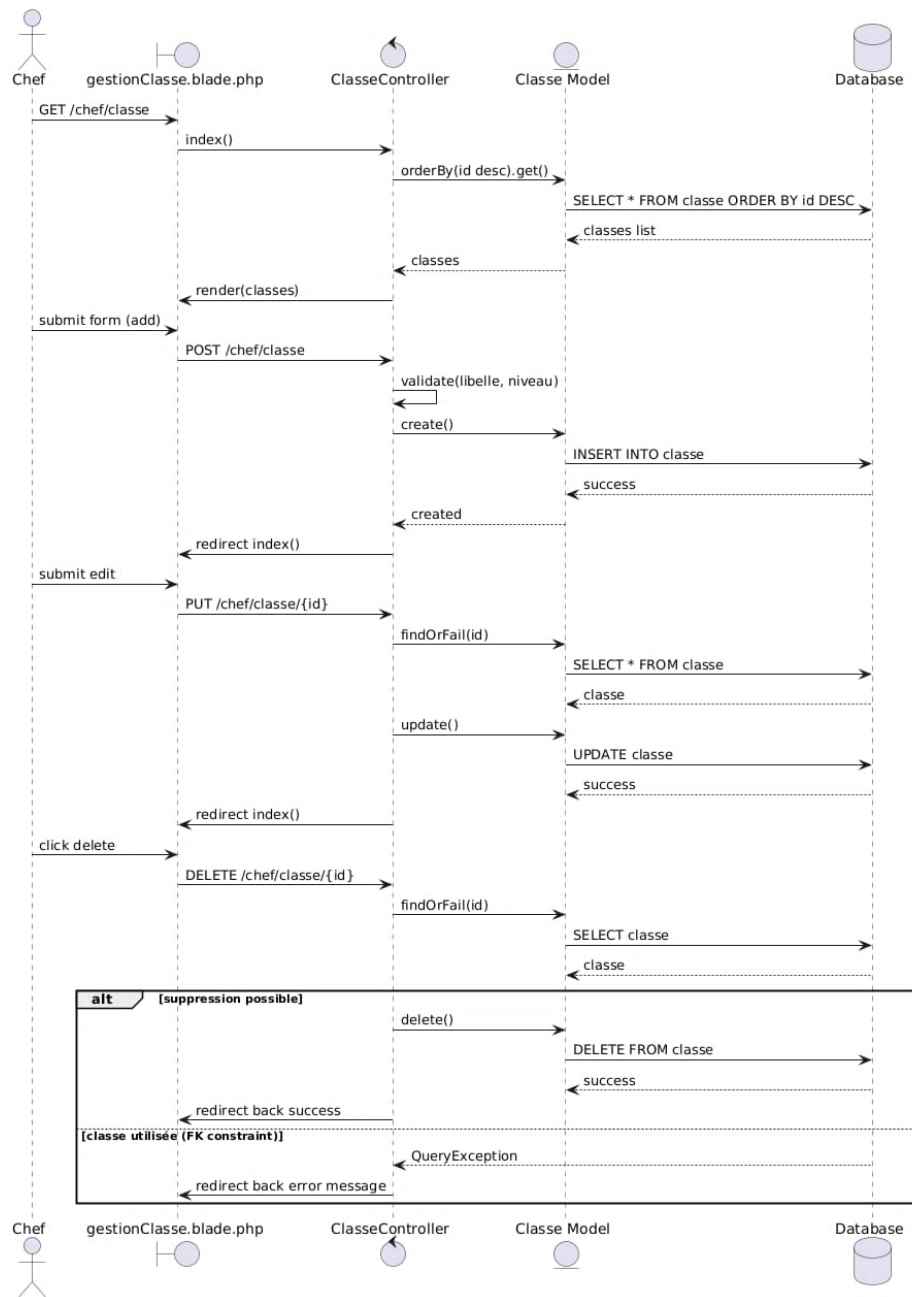


Figure 6: Diagramme de séquence — Gestion des classes

Diagramme de séquence — Gestion des séances (Figure 6) : Ce diagramme est l'un des plus complexes du système. Il modélise l'ensemble du cycle CRUD des séances, ainsi que le

mécanisme de vérification de disponibilité. Lorsque l'utilisateur sélectionne une date et un horaire, des requêtes asynchrones sont envoyées pour récupérer les classes, salles et enseignants disponibles (non déjà assignés à une séance sur ce créneau).

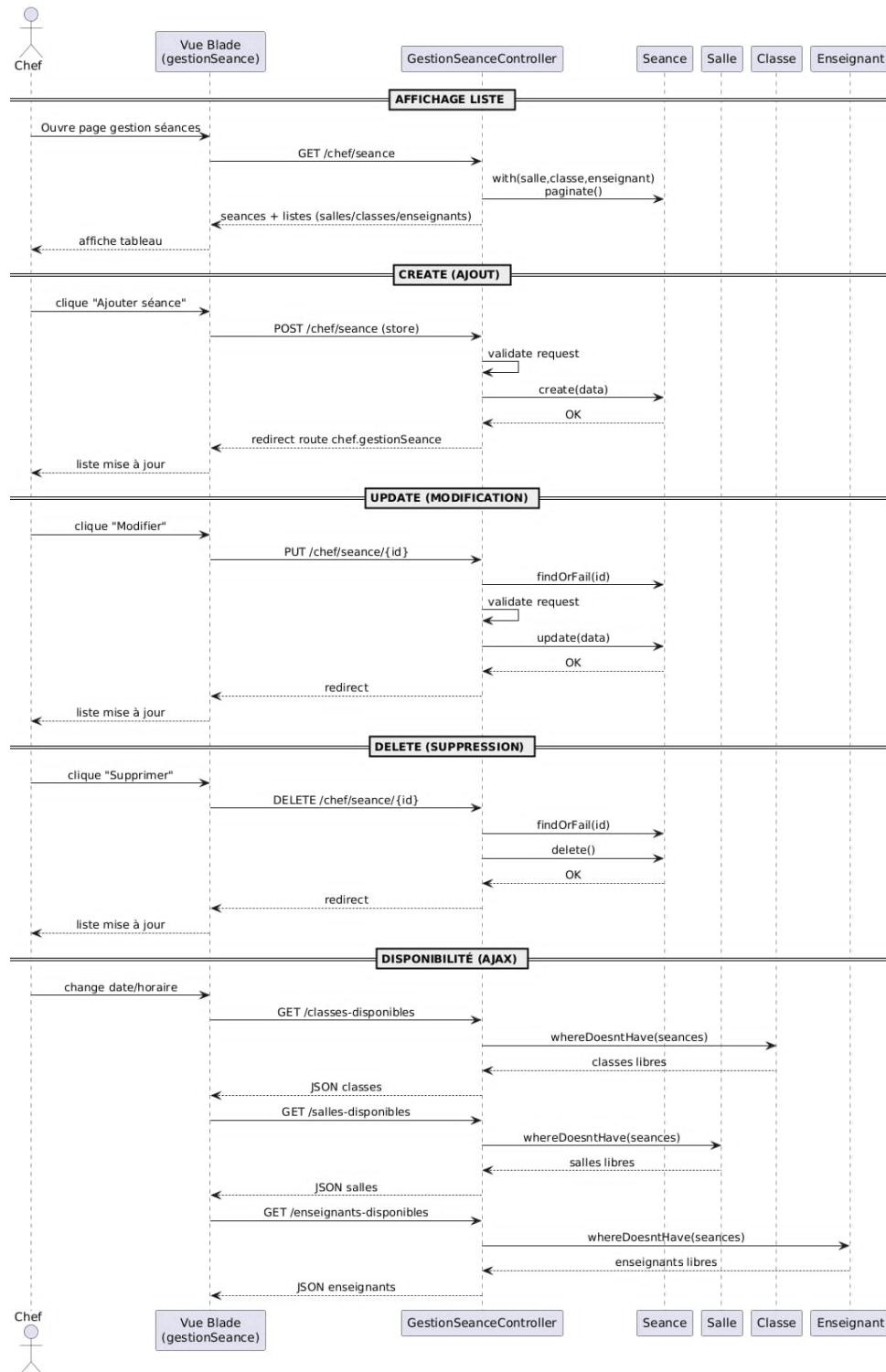


Figure 7: Diagramme de séquence — Gestion des séances

Diagramme de séquence — Réservations enseignant (Figure 7) : Ce diagramme montre le processus complet de réservation d'une salle par un enseignant, incluant la vérification des conflits (un enseignant, une classe ou une salle ne peut être assignée à deux séances simultanées), et le traitement différencié selon qu'un conflit est détecté ou non.

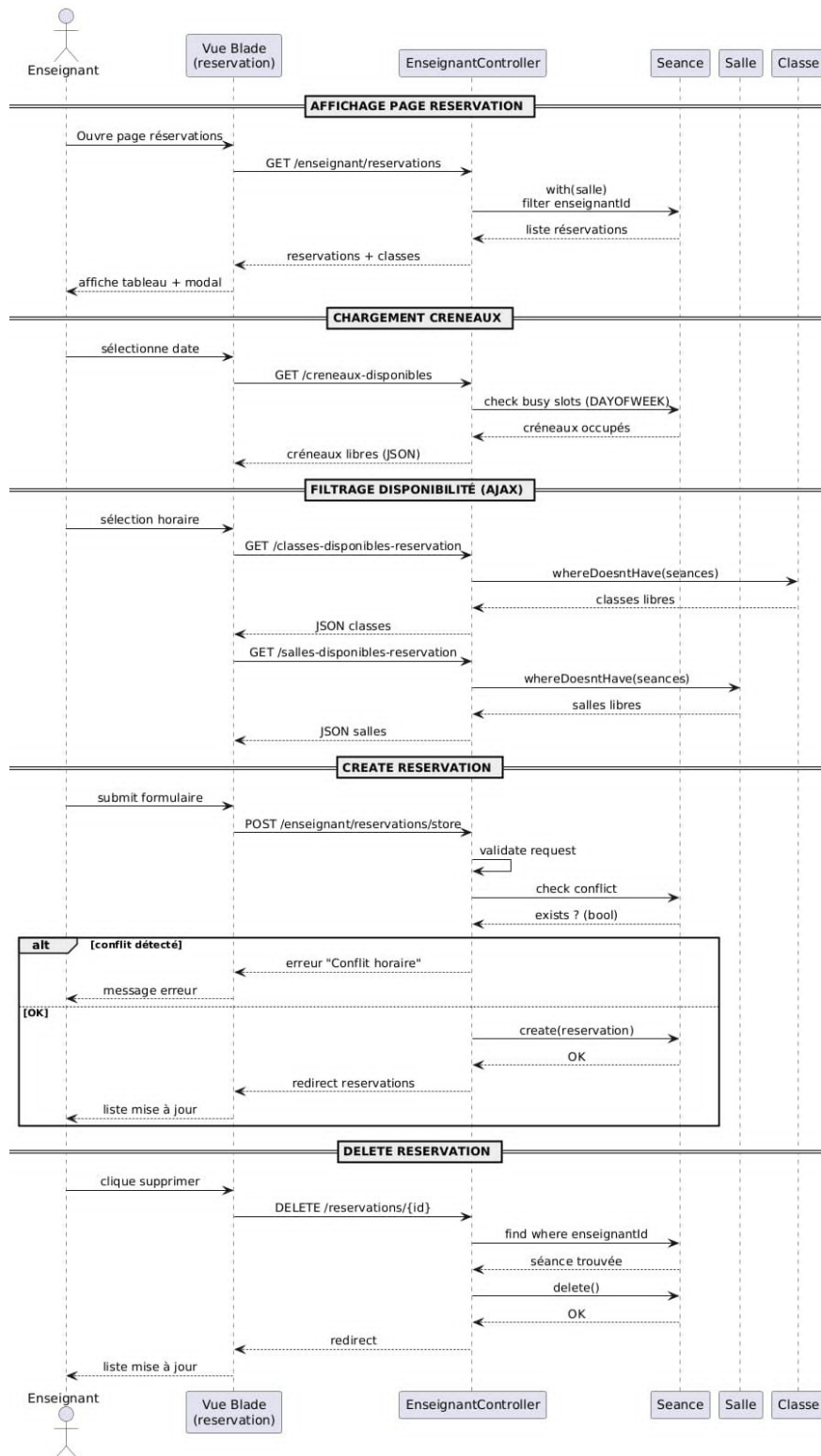


Figure 8: Diagramme de séquence — Réservations enseignant

2.5 Conception de la base de données

La base de données d'UniSalles est une base de données relationnelle MySQL dont la conception reflète le modèle de classes présenté précédemment. Elle comprend les tables suivantes :

- **users** : stocke les informations d'authentification et le rôle de tous les utilisateurs (id, name, email, password, role).
- **enseignants** : profil étendu des enseignants, lié à la table users par une clé étrangère (id, user_id, nom, prenom).
- **etudiants** : profil des étudiants avec référence à leur classe (id, user_id, classe_id, nom, prenom).
- **salles** : informations sur les salles (id, nomSalle, capacite, type, disponibilite).
- **classes** : informations sur les promotions (id, libelle, niveau).
- **seances** : planification des séances avec références vers les tables enseignants, salles, classes (id, matiere, date, heureDebut, heureFin, enseignant_id, salle_id, classe_id).
- **reclamations** : réclamations soumises par les enseignants (id, titre, description, type, statut, enseignant_id).
- **messages** : messages envoyés aux étudiants (id, message, enseignant_id, classe_id).

Les clés étrangères assurent l'intégrité référentielle entre les tables. Les index appropriés sont définis sur les colonnes fréquemment utilisées dans les clauses WHERE et JOIN pour optimiser les performances des requêtes.

Ce deuxième chapitre a permis de poser les bases conceptuelles solides sur lesquelles repose le développement d'UniSalles. La prochaine étape consiste à traduire cette conception en une implémentation technique concrète.

Chapitre 3 : Réalisation technique

Ce troisième chapitre est consacré aux aspects techniques du projet UniSalles. Il décrit l'architecture générale de l'application, les technologies retenues et les raisons de ces choix, les modalités de mise en œuvre de la sécurité et de l'authentification, ainsi que la structure de la base de données.

3.1 Architecture générale

UniSalles est une application web monolithique développée selon le patron architectural MVC (Modèle-Vue-Contrôleur). Cette architecture, nativement supportée par le framework Laravel, organise le code de l'application en trois couches distinctes et complémentaires, favorisant la séparation des préoccupations et la maintenabilité du code.

L'application est structurée selon les conventions de Laravel, avec les dossiers principaux suivants : `app/Models` pour les modèles Eloquent, `app/Http/Controllers` pour les contrôleurs, `resources/views` pour les vues Blade, `routes/web.php` pour la définition des routes, et `database/migrations` pour les scripts de migration de la base de données.

3.2 Architecture MVC

3.2.1 Le Modèle (Model)

La couche Modèle est implémentée à l'aide de l'ORM Eloquent, inclus nativement dans Laravel. Chaque entité métier du système (User, Salle, Classe, Seance, Reclamation, Message, Enseignant, Etudiant) est représentée par une classe PHP héritant de la classe Model d'Eloquent. Ces classes définissent les attributs fillables (protégés contre l'assignation de masse), les relations entre entités (`hasOne`, `hasMany`, `belongsTo`, `belongsToMany`) et les règles de validation éventuelles.

L'ORM Eloquent permet d'interagir avec la base de données en utilisant une syntaxe orientée objet, sans écrire de requêtes SQL brutes dans la grande majorité des cas, ce qui améliore la lisibilité du code et réduit les risques d'injection SQL.

3.2.2 La Vue (View)

La couche Vue est implementée avec le moteur de templates Blade de Laravel. Les vues Blade permettent d'écrire du HTML enrichi de directives spécifiques (@if, @foreach, @extends, @section, @yield, etc.) qui facilitent la création d'une interface dynamique tout en maintenant une séparation claire entre la logique de présentation et la logique métier.

L'architecture des vues repose sur un template de base (layout) qui définit la structure commune à toutes les pages de l'application (en-tête, menu de navigation, pied de page). Les vues spécifiques à chaque fonctionnalité étendent ce template et n'en redéfinissent que le contenu central.

3.2.3 Le Contrôleur (Controller)

Les contrôleurs constituent la couche de liaison entre les modèles et les vues. Ils reçoivent les requêtes HTTP, appliquent la logique métier appropriée en s'appuyant sur les modèles, puis retournent une réponse au client (sous forme de vue rendue, de redirection ou de réponse JSON pour les requêtes AJAX).

UniSalles utilise des Resource Controllers de Laravel, qui fournissent une implémentation standard des sept méthodes CRUD (index, create, store, show, edit, update, destroy) pour chaque entité gérée. Cette approche normalise la structure du code et facilite sa compréhension.

3.3 Technologies utilisées

Le tableau suivant récapitule les technologies retenues pour le développement d'UniSalles ainsi que les raisons de leur sélection :

Composant	Technologie	Version / Détail
Framework back-end	Laravel (PHP)	PHP 8.x / Laravel 10.x
Moteur de templates	Blade	Intégré à Laravel
Framework CSS	Bootstrap	Bootstrap 5
Langage client	JavaScript / jQuery	ES6 + jQuery 3.x
Base de données	MySQL	MySQL 8.x
Gestionnaire de BDD	phpMyAdmin	Interface web
Environnement local	Laragon	Apache + PHP + MySQL
IDE	Visual Studio Code	avec extensions PHP
Gestion de version	Git / GitHub	Repository GitHub
Architecture	MVC	Patron Laravel natif

3.3.1 Justification des choix technologiques

Laravel a été choisi comme framework back-end pour plusieurs raisons déterminantes. Il propose une architecture MVC robuste et bien documentée, un ORM puissant (Eloquent), un système d'authentification prêt à l'emploi, un moteur de templates (Blade) expressif, et un écosystème riche facilitant le développement rapide d'applications web sécurisées. Sa large communauté et l'abondance de ressources pédagogiques disponibles ont également pesé dans la décision.

Bootstrap 5 a été retenu pour la conception de l'interface utilisateur en raison de sa simplicité d'utilisation, de sa richesse en composants prédéfinis (modales, tableaux, formulaires, alertes) et de sa conception mobile-first garantissant une expérience utilisateur cohérente sur tous les types d'appareils.

MySQL, en tant que système de gestion de bases de données relationnelles, offre des performances éprouvées, une compatibilité native avec Laravel/Eloquent et une grande facilité d'administration via phpMyAdmin.

3.4 Structure de la base de données

La base de données MySQL d'UniSalles est composée de huit tables principales dont les structures sont les suivantes :

Table users

Champs : id (bigint, PK, auto-increment), name (varchar 255), email (varchar 255, unique), email_verified_at (timestamp, nullable), password (varchar 255, hashé), role (enum : 'chef', 'enseignant', 'etudiant'), remember_token (varchar 100, nullable), created_at / updated_at (timestamps).

Table enseignants

Champs : id (bigint, PK), user_id (bigint, FK → users.id, cascade delete), nom (varchar 255), prenom (varchar 255), created_at / updated_at.

Table etudiants

Champs : id (bigint, PK), user_id (bigint, FK → users.id), classe_id (bigint, FK → classes.id), nom (varchar 255), prenom (varchar 255), created_at / updated_at.

Table salles

Champs : id (bigint, PK), nomSalle (varchar 255), capacite (int), type (varchar 255), disponibilite (boolean, défaut 1), created_at / updated_at.

Table classes

Champs : id (bigint, PK), libelle (varchar 255), niveau (int), created_at / updated_at.

Table seances

Champs : id (bigint, PK), matiere (varchar 255), date (date), heureDebut (time), heureFin (time), enseignant_id (bigint, FK → enseignants.id), salle_id (bigint, FK → salles.id), classe_id (bigint, FK → classes.id), created_at / updated_at.

Table reclamations

Champs : id (bigint, PK), titre (varchar 255), description (text), type (varchar 255), statut (enum : 'en attente', 'en cours', 'résolu'), enseignant_id (bigint, FK → enseignants.id), created_at / updated_at.

Table messages

Champs : id (bigint, PK), message (text), enseignant_id (bigint, FK → enseignants.id), classe_id (bigint, FK → classes.id), created_at / updated_at.

3.5 Sécurité et authentification

3.5.1 Système d'authentification

UniSalles utilise le système d'authentification natif de Laravel, qui repose sur les sessions PHP sécurisées. Le processus de connexion est géré par l'AuthenticatedSessionController, qui exploite le LoginRequest pour valider les données d'entrée et le façade Auth de Laravel pour authentifier l'utilisateur.

Les mots de passe sont stockés dans la base de données sous forme de hash bcrypt, un algorithme de hachage cryptographique conçu pour être volontairement lent afin de résister aux attaques par force brute. Laravel effectue ce hachage automatiquement lors de la création d'un compte utilisateur.

Lors de la connexion, Laravel régénère l'identifiant de session après authentification réussie pour prévenir les attaques par fixation de session (session fixation attacks). La déconnexion détruit complètement la session côté serveur et invalide le token CSRF.

3.5.2 Gestion des rôles et des accès

La gestion des rôles est implémentée via un système de middlewares personnalisés. Chaque groupe de routes est protégé par un ou plusieurs middlewares qui vérifient, d'une part, que l'utilisateur est authentifié (middleware auth), et d'autre part, qu'il dispose du rôle requis pour accéder aux ressources demandées.

3.5.3 Protection CSRF

Laravel génère automatiquement un token CSRF (Cross-Site Request Forgery) pour chaque session utilisateur. Ce token est inclus dans tous les formulaires HTML de l'application via la directive Blade `@csrf`. Lors du traitement des requêtes POST, PUT et DELETE, Laravel vérifie la présence et la validité de ce token, rejetant toute requête qui ne le contient pas ou dont le token est invalide. Cette protection prévient les attaques CSRF qui pourraient permettre à un site malveillant d'effectuer des actions au nom d'un utilisateur authentifié.

Ce troisième chapitre a détaillé l'ensemble des choix techniques qui sous-tendent le développement d'UniSalles. L'architecture retenue garantit à la fois la robustesse, la sécurité et la maintenabilité de l'application, tout en permettant une évolution progressive des fonctionnalités.

Chapitre 4 : Présentation des interfaces et fonctionnalités

Ce quatrième chapitre présente les différentes interfaces de la plateforme UniSalles et décrit les fonctionnalités qu'elles exposent. Organisé par modules fonctionnels, il offre une vue d'ensemble complète de l'application du point de vue de l'utilisateur.

4.1 Module d'authentification

4.1.1 Page de connexion

La page de connexion est la porte d'entrée de la plateforme UniSalles. Elle présente un formulaire épuré demandant l'adresse e-mail et le mot de passe de l'utilisateur. Le design, réalisé avec Bootstrap 5, est moderne et responsive. Un message d'erreur s'affiche automatiquement en cas d'identifiants incorrects.

Après validation réussie des identifiants, le système identifie le rôle de l'utilisateur et le redirige automatiquement vers le tableau de bord correspondant : chef.dashboard pour le chef de département, enseignant.dashboard pour un enseignant, ou etudiant.dashboard pour un étudiant. Cette redirection différenciée constitue le mécanisme central de la gestion des rôles côté utilisateur.

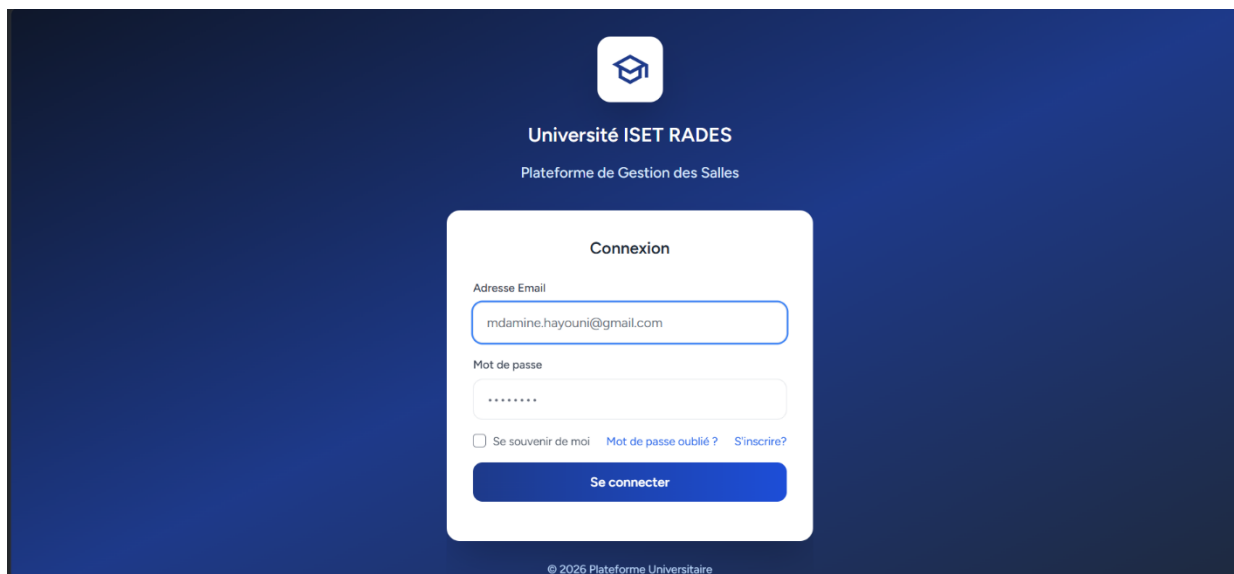


Figure 9:Page de Connexion

4.1.2 Page d'inscription (étudiants)

La page d'inscription est accessible aux nouveaux étudiants souhaitant créer un compte. Elle collecte les informations nécessaires à la création simultanée d'un compte User et d'un profil Etudiant : nom complet, prénom, adresse e-mail, classe d'appartenance (sélectionnable dans une liste déroulante chargée dynamiquement), et mot de passe avec confirmation.

L'inscription est traitée dans une transaction de base de données pour garantir l'atomicité : soit les deux enregistrements (User et Etudiant) sont créés avec succès, soit aucun ne l'est en cas d'erreur. À l'issue de l'inscription, l'étudiant est automatiquement connecté et redirigé vers son tableau de bord.

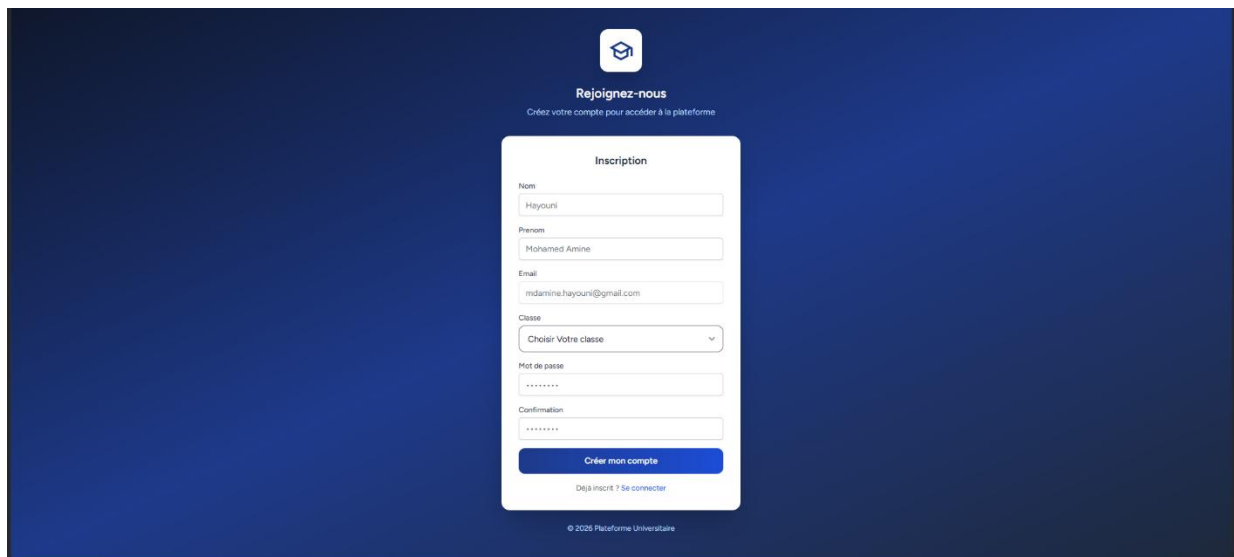


Figure 10:Page d'Inscription

4.2 Module Chef de département

4.2.1 Tableau de bord

Le tableau de bord du chef de département est la première page affichée après la connexion. Il présente un ensemble de widgets statistiques permettant d'avoir une vue d'ensemble instantanée de l'état de l'établissement. Ces widgets affichent notamment le nombre total de salles enregistrées dans le système, le nombre de réclamations en attente de traitement, et le nombre total d'enseignants.

Ces statistiques sont calculées en temps réel à chaque chargement de la page via des requêtes Eloquent optimisées (count()), comme illustré dans le diagramme de séquence du Dashboard. La navigation vers les différents modules de gestion est accessible depuis un menu latéral persistant.

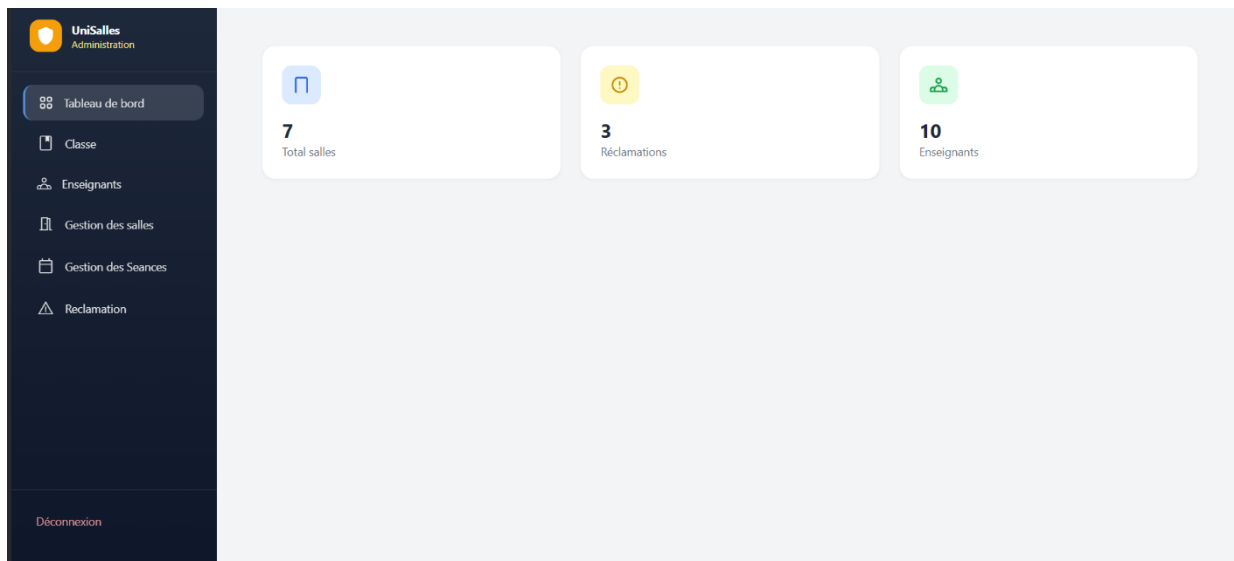
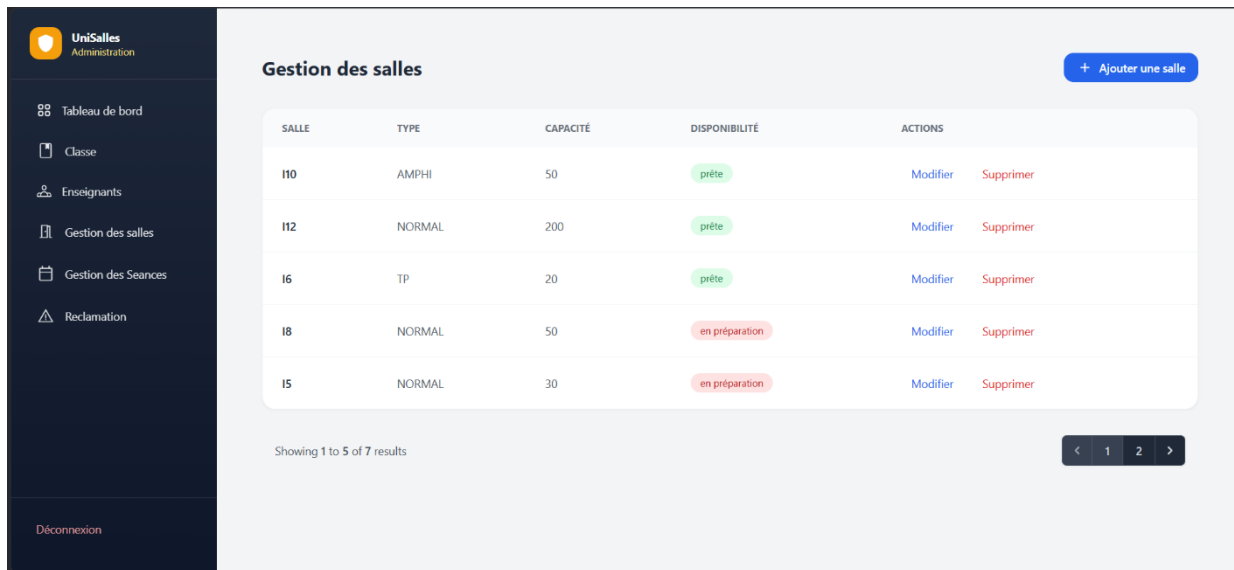


Figure 11: Dashboard Chef département

4.2.2 Gestion des salles

Le module de gestion des salles permet au chef de département d'effectuer l'ensemble des opérations CRUD sur les salles de l'établissement. La liste des salles est affichée sous forme de tableau paginé (5 éléments par page) présentant pour chaque salle son nom, sa capacité, son type et sa disponibilité.

L'ajout et la modification d'une salle s'effectuent via des formulaires présentés dans des fenêtres modales Bootstrap, ce qui évite la navigation vers une page dédiée et améliore la fluidité de l'expérience utilisateur. La suppression est protégée par une confirmation avant exécution.



Gestion des salles [+ Ajouter une salle](#)

SALLE	TYPE	CAPACITÉ	DISPONIBILITÉ	ACTIONS
I10	AMPHI	50	prête	Modifier Supprimer
I12	NORMAL	200	prête	Modifier Supprimer
I6	TP	20	prête	Modifier Supprimer
I8	NORMAL	50	en préparation	Modifier Supprimer
I5	NORMAL	30	en préparation	Modifier Supprimer

Showing 1 to 5 of 7 results

< 1 2 >

Déconnexion

Figure 12: Interface Gestion Salles

4.2.3 Gestion des classes

La gestion des classes (promotions d'étudiants) suit une approche similaire à celle des salles. Le chef de département peut créer, modifier et supprimer des classes. Chaque classe est caractérisée par un libellé (ex. : « Licence Informatique 3 ») et un niveau.

Une protection particulière est mise en place lors de la suppression : si une classe est référencée par des étudiants ou des séances, la suppression est bloquée et un message d'erreur explicatif est affiché. Cette protection est implémentée via la capture de l'exception `QueryException` levée par MySQL lorsqu'une contrainte de clé étrangère est violée.

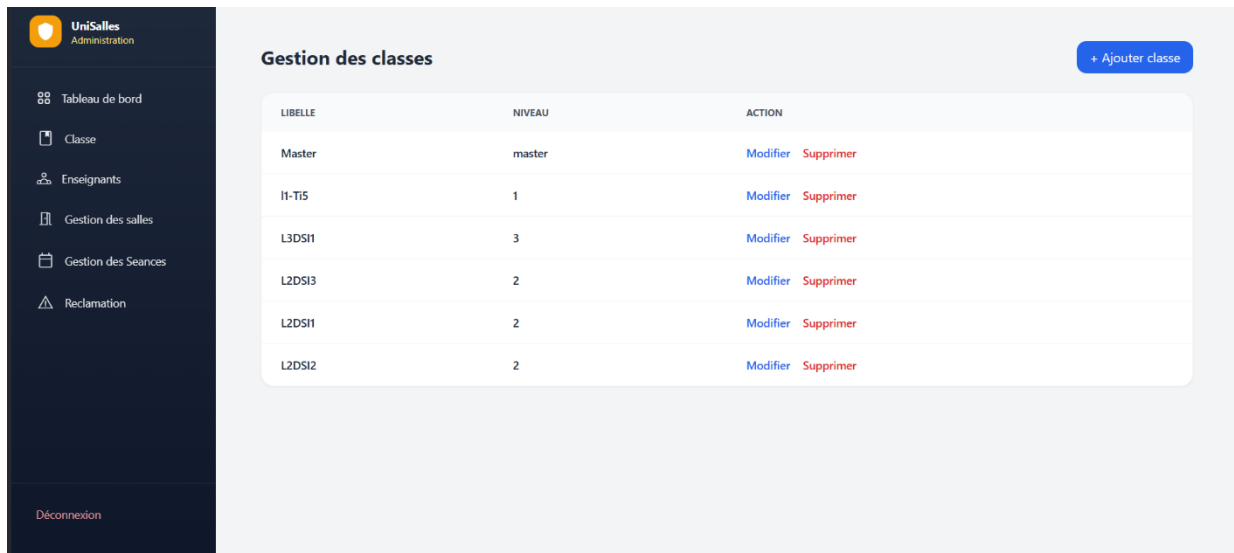


Figure 13: Interface Gestion Classes

4.2.4 Gestion des séances

Le module de gestion des séances est le plus complexe de l'application. Il permet au chef de département de planifier des séances de cours en affectant à chaque séance une matière, une date, un créneau horaire (heure de début et heure de fin), un enseignant, une salle et une classe.

La particularité de ce module réside dans son système de vérification des disponibilités en temps réel. Lorsque l'utilisateur sélectionne une date et un horaire dans le formulaire, des requêtes sont automatiquement envoyées au serveur pour récupérer les listes dynamiques des classes, des salles et des enseignants disponibles sur ce créneau (c'est-à-dire non déjà assignés à une autre séance). Les listes déroulantes correspondantes sont mises à jour instantanément sans rechargement de la page, offrant ainsi une expérience utilisateur fluide et réduisant le risque de conflits de planification.

Gestion des séances
Ajouter et gérer les séances de cours

+ Ajouter une séance

MATIÈRE	DATE	HORAIRE	ENSEIGNANT(E)	CLASSE	SALLE	ACTIONS
math	2026-05-04	10:15:00 - 11:45:00	hayouni	L2DSI2	I10	Modifier Supprimer
SID	2026-05-05	08:30:00 - 10:00:00	Ksouri	L2DSI2	I12	Modifier Supprimer
SID	2026-05-05	10:15:00 - 11:45:00	hayouni	L2DSI1	I10	Modifier Supprimer
Francais	2026-05-12	13:30:00 - 15:00:00	zghonda	L2DSI2	I5	Modifier Supprimer
Anglais	2026-05-08	08:30:00 - 10:00:00	KHECHINI	L2DSI2	I8	Modifier Supprimer
Java	2026-05-11	08:30:00 - 10:00:00	hayouni	L2DSI2	I12	Modifier Supprimer
AT java	2026-05-11	10:15:00 - 11:45:00	mokrani	L2DSI2	I12	Modifier Supprimer

Showing 1 to 7 of 19 results

< 1 2 3 >

Figure 14: Interface Gestion séances

4.2.5 Gestion des réclamations

Le chef de département dispose d'une interface dédiée au traitement des réclamations soumises par les enseignants. Cette interface liste l'ensemble des réclamations avec leur titre, leur type, leur statut actuel et la date de soumission. Il peut modifier le statut d'une réclamation (en attente → en cours → résolu) pour informer l'enseignant concerné de l'avancement du traitement de sa demande.

Réclamations

Probleme nettoyage
Signalé par: Aouadi
Salle: I3
La salle non propre
51 minutes ago [Traiter](#)

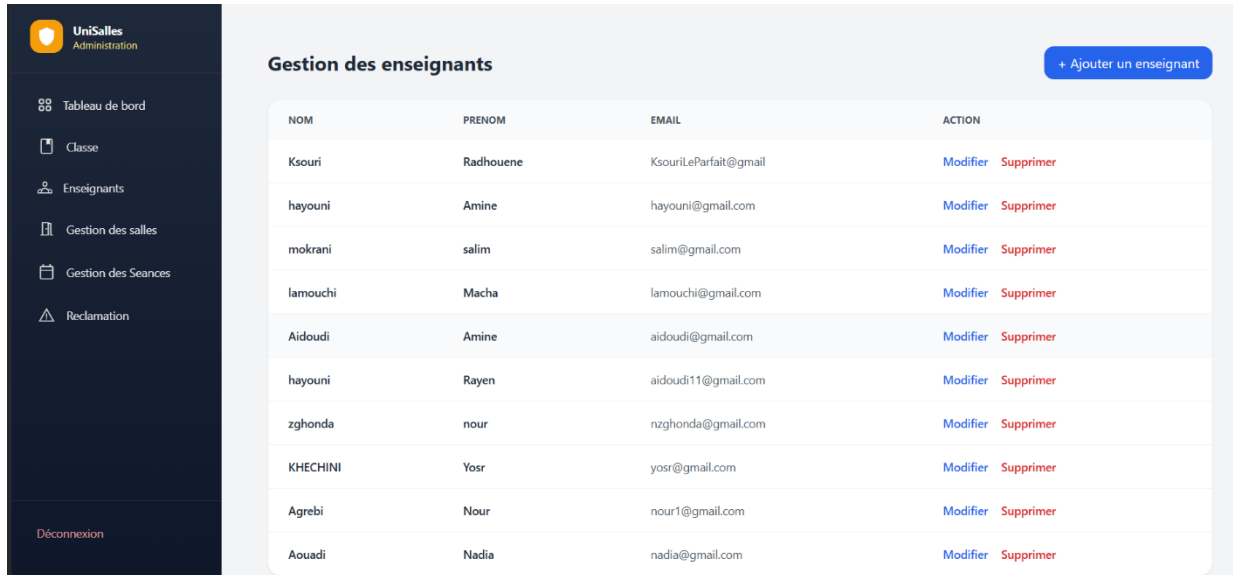
salle endommagé
Signalé par: KHECHINI
Salle: I6
sdtghjklmjhgfds
2 days ago [Traiter](#)

Projecteur en Panne
Signalé par: KHECHINI
Salle: I12
Merci de remplacer le projecteur!
2 days ago [Traiter](#)

Figure 15: Interface Gestion des Réclamations

4.2.6 Gestion des enseignants

Ce module permet au chef de département de créer, modifier et supprimer des comptes enseignants. La création d'un enseignant entraîne la création simultanée d'un compte User (avec le rôle 'enseignant') et d'un profil Enseignant associé, selon un processus similaire à l'inscription des étudiants.



NOM	PRENOM	EMAIL	ACTION
Ksouri	Radhouene	KsouriLeParfait@gmail	Modifier Supprimer
hayouni	Amine	hayouni@gmail.com	Modifier Supprimer
mokrani	salim	salim@gmail.com	Modifier Supprimer
lamouchi	Macha	lamouchi@gmail.com	Modifier Supprimer
Aidoudi	Amine	aidoudi@gmail.com	Modifier Supprimer
hayouni	Rayen	aidoudi11@gmail.com	Modifier Supprimer
zghonda	nour	nzghonda@gmail.com	Modifier Supprimer
KHECHINI	Yosr	yosr@gmail.com	Modifier Supprimer
Agrebi	Nour	nour1@gmail.com	Modifier Supprimer
Aouadi	Nadia	nadia@gmail.com	Modifier Supprimer

Figure 16: Gestion des Enseignants

4.3 Module Enseignant

4.3.1 Tableau de bord enseignant

Le tableau de bord de l'enseignant lui présente un résumé de ses activités sur la plateforme : ses séances planifiées à venir, ses réservations récentes et les statuts de ses réclamations. Il constitue le point d'entrée principal de l'espace enseignant.

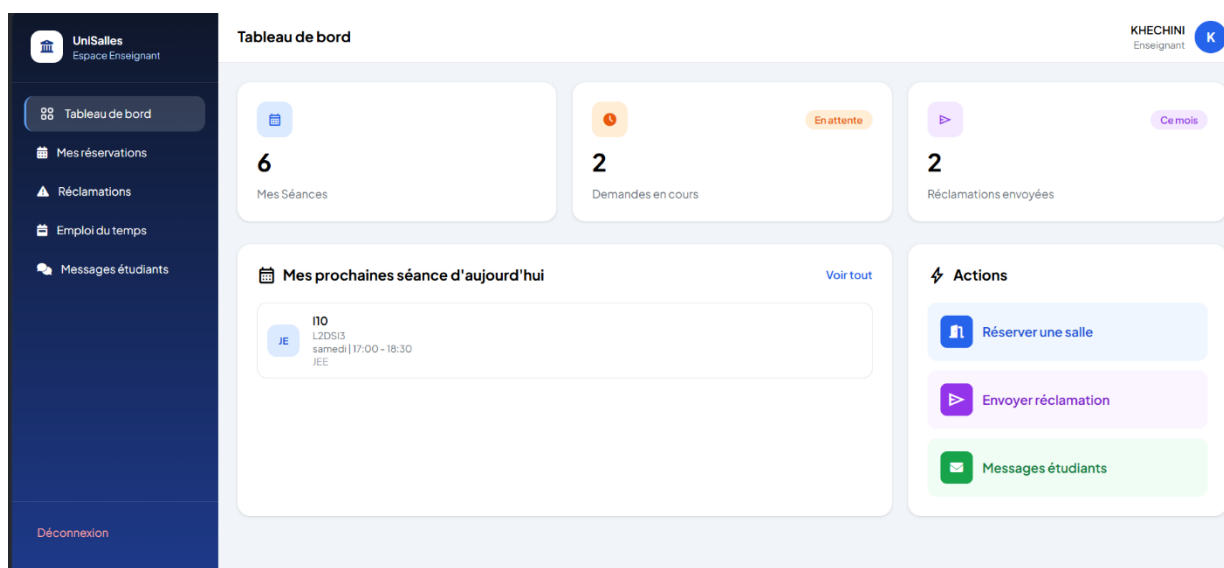


Figure 17:Dashboard Enseignant

4.3.2 Réservation de salles

Le module de réservation permet à l'enseignant de réserver une salle pour une séance non encore planifiée. Le formulaire de réservation propose une sélection de la date, du créneau horaire (avec chargement dynamique des créneaux disponibles via AJAX selon la date choisie), de la classe et de la salle.

Lors de la soumission du formulaire, le système vérifie l'absence de conflits : ni l'enseignant, ni la salle, ni la classe ne doivent être déjà assignés à une autre séance sur le créneau demandé. En cas de conflit détecté, un message d'erreur explicatif est affiché. En l'absence de conflit, la réservation est enregistrée et l'enseignant est redirigé vers la liste de ses réservations mise à jour. L'enseignant peut également annuler une réservation existante depuis la liste de ses réservations, à condition que la séance concernée ne soit pas encore passée.

Salle	Matière	Jour	Horaire	Action
I10	Math	lundi	08:30 - 10:00	Supprimer
I8	Anglais	vendredi	08:30 - 10:00	Supprimer
I10	English	samedi	08:30 - 10:00	Supprimer
I10	JEE	samedi	17:00 - 18:30	Supprimer

Figure 18: Interface Réservation Salle

4.3.3 Envoi de réclamations

L'enseignant peut soumettre des réclamations au chef de département depuis un formulaire dédié. Il renseigne le titre de la réclamation, sa description détaillée et son type (problème matériel, absence justifiée, demande de rattrapage, etc.). La réclamation est enregistrée avec le statut initial « en attente » et devient visible dans l'interface du chef de département.

Figure 19: Interface Envoi de Réclamations

4.3.4 Consultation de l'emploi du temps

L'enseignant peut consulter son emploi du temps personnel, affichant l'ensemble des séances auxquelles il est assigné, triées par date et heure. Chaque séance affiche la matière, la salle, la classe et le créneau horaire.

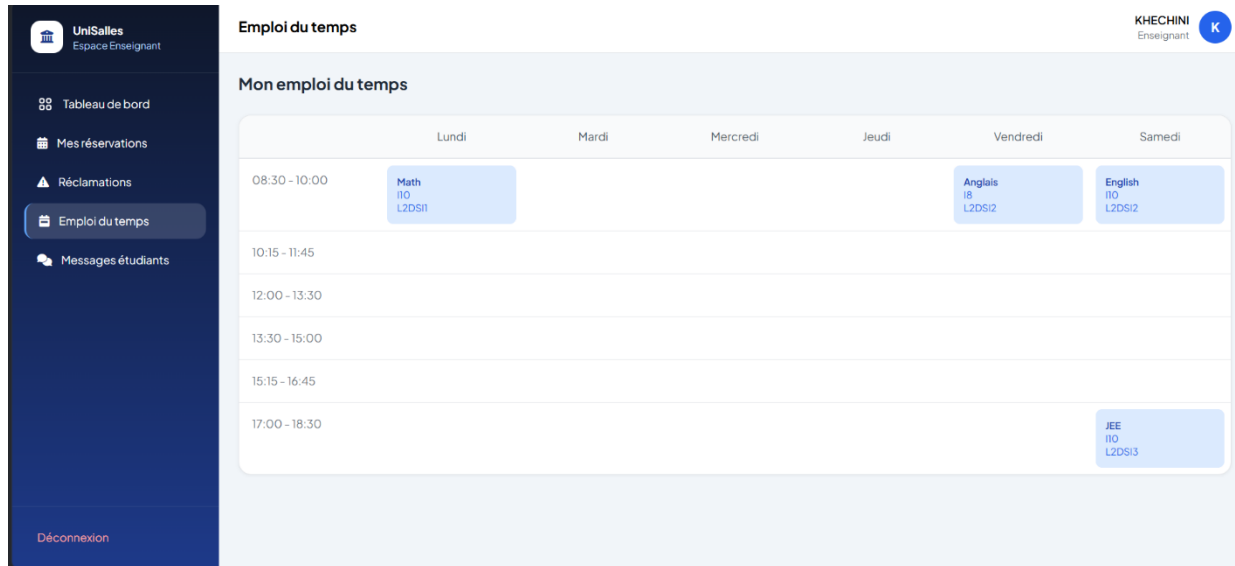


Figure 20: Interface Emploi du temps

4.4 Module Étudiant

4.4.1 Tableau de bord étudiant

Le tableau de bord de l'étudiant est conçu pour fournir un accès rapide et clair aux informations les plus pertinentes : l'emploi du temps de sa classe et les notifications éventuelles envoyées par le chef de département.

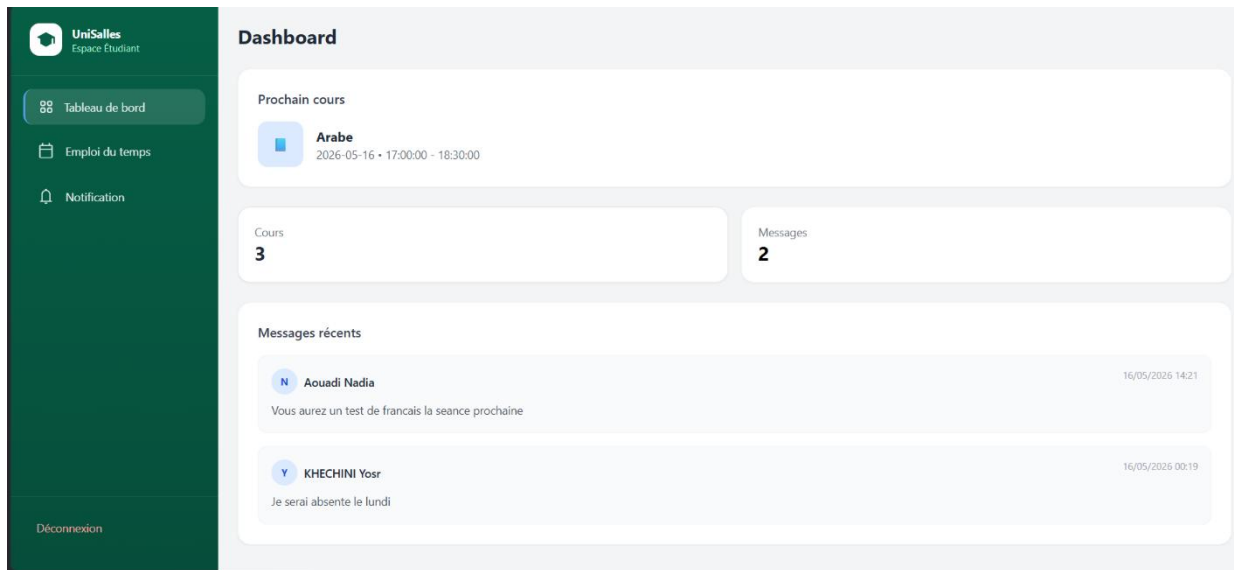


Figure 21: Dashboard Étudiant

4.4.2 Consultation de l'emploi du temps

L'étudiant peut consulter l'emploi du temps de sa classe, affichant toutes les séances planifiées avec leurs détails (matière, enseignant, salle, date et horaire). Cette vue lui permet d'anticiper les cours à venir et d'être informé de tout changement.

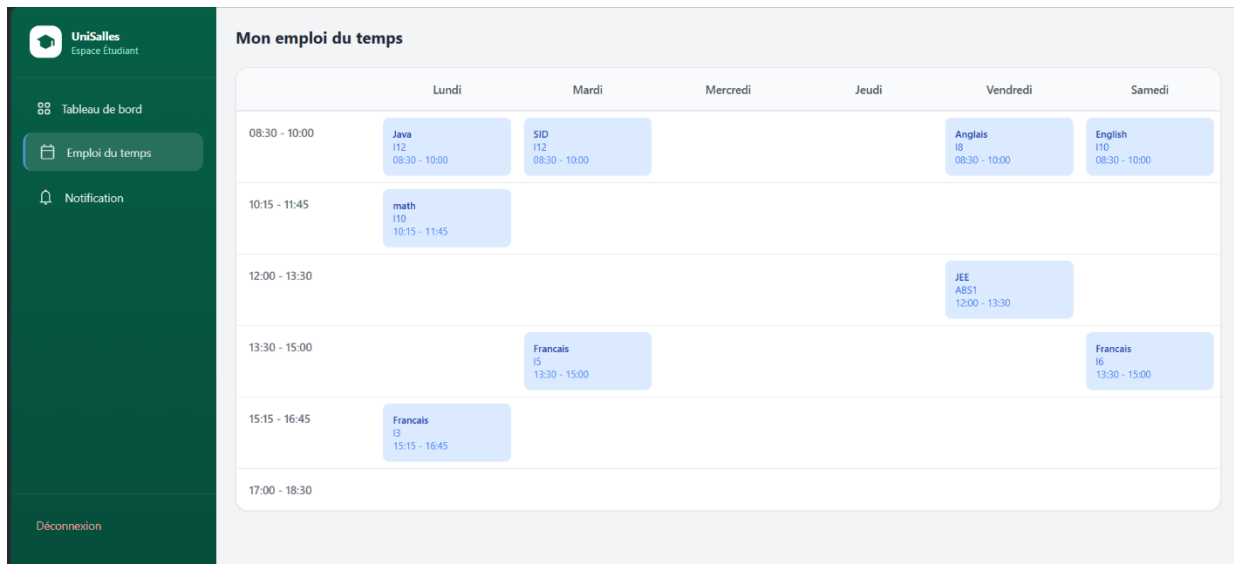


Figure 22: Interface Emploi du temps (étudiant)

4.4.3 Notifications

Les messages envoyés par le chef de département à la classe de l'étudiant sont affichés dans un espace notifications dédié. Cela permet une communication directe et centralisée entre l'administration et les étudiants, notamment pour les annonces de rattrapages, changements de salle ou informations importantes.

Ce quatrième chapitre a présenté l'ensemble des fonctionnalités d'UniSalles telles qu'elles sont accessibles aux utilisateurs finaux. La richesse et la cohérence de ces fonctionnalités témoignent de la qualité de la phase de conception et de la solidité des choix techniques effectués.

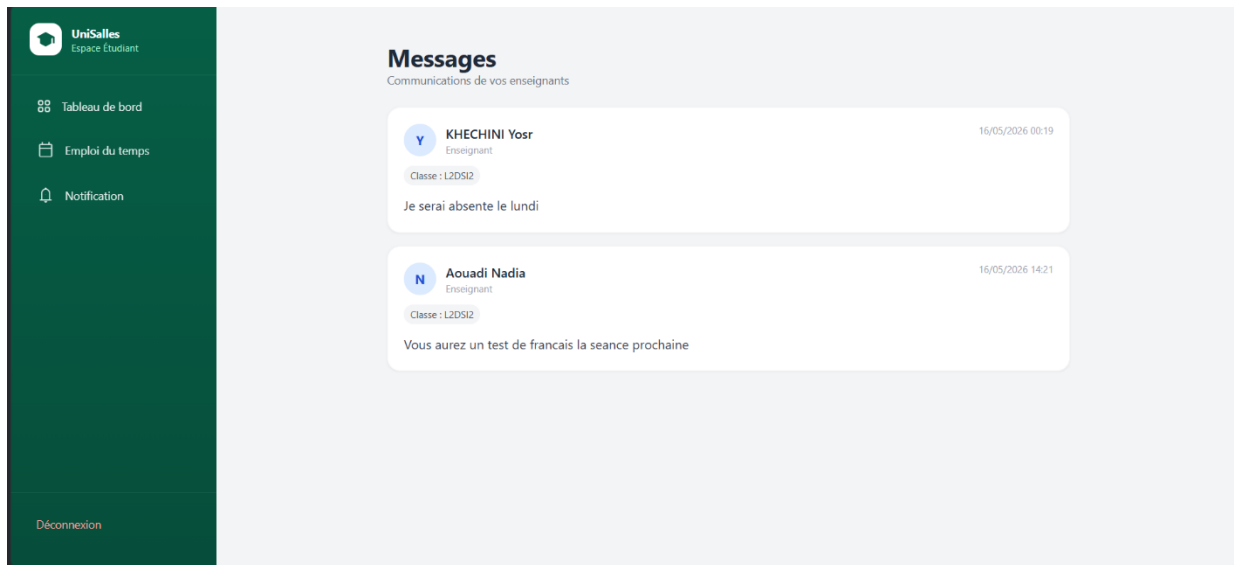


Figure 23: Interface Notification(etudiant)

Chapitre 5 : Tests et validation

Ce cinquième et dernier chapitre est consacré à la démarche de test et de validation de la plateforme UniSalles. Il présente les tests fonctionnels réalisés, les résultats obtenus, les difficultés rencontrées au cours du développement et les solutions apportées, avant de conclure par les perspectives d'amélioration envisagées pour la suite du projet.

5.1 Tests fonctionnels

La validation d'UniSalles a été conduite principalement par des tests fonctionnels manuels, réalisés tout au long du développement selon une approche itérative conforme à la méthodologie Agile Scrum adoptée. Chaque fonctionnalité développée a été testée immédiatement après son implémentation, puis retestée lors de l'intégration avec les autres modules.

Le tableau suivant synthétise les principaux cas de test réalisés et leurs résultats :

Cas de test	Description	Résultat
CT-01	Connexion avec identifiants valides (chef)	PASS
CT-02	Connexion avec identifiants valides (enseignant)	PASS
CT-03	Connexion avec identifiants valides (étudiant)	PASS
CT-04	Connexion avec identifiants incorrects	PASS
CT-05	Inscription d'un étudiant avec données valides	PASS
CT-06	Inscription avec e-mail déjà utilisé	PASS
CT-07	Création d'une salle (chef)	PASS
CT-08	Modification d'une salle (chef)	PASS
CT-09	Suppression d'une salle (chef)	PASS
CT-10	Création d'une classe (chef)	PASS
CT-11	Suppression d'une classe référencée (FK)	PASS
CT-12	Planification d'une séance sans conflit	PASS
CT-13	Planification d'une séance avec conflit de salle	PASS
CT-14	Chargement AJAX des disponibilités	PASS
CT-15	Réservation d'une salle par un enseignant	PASS
CT-16	Réservation en double (même créneau)	PASS

Cas de test	Description	Résultat
CT-17	Soumission d'une réclamation	PASS
CT-18	Traitement d'une réclamation (chef)	PASS
CT-19	Consultation emploi du temps (enseignant)	PASS
CT-20	Consultation emploi du temps (étudiant)	PASS
CT-21	Envoi de notification aux étudiants	PASS
CT-22	Accès non autorisé à une route protégée	PASS
CT-23	Protection CSRF sur les formulaires	PASS
CT-24	Validation des données côté serveur	PASS

5.2 Résultats obtenus

L'ensemble des vingt-quatre cas de test réalisés ont produit les résultats attendus. Les fonctionnalités principales de la plateforme fonctionnent correctement : l'authentification différenciée selon les rôles redirige bien chaque utilisateur vers son espace dédié ; les opérations CRUD sur les salles et les classes respectent les règles de gestion définies (notamment la protection contre la suppression de classes référencées) ; le mécanisme de vérification des disponibilités via AJAX fonctionne en temps réel ; le système de détection des conflits de réservation rejette correctement les tentatives de double réservation ; et les protections de sécurité (authentification, rôles, CSRF) opèrent comme prévu.

L'interface s'est révélée fonctionnelle sur les principaux navigateurs testés (Chrome, Firefox, Edge) et sur différentes tailles d'écran (ordinateur de bureau, tablette), conformément aux objectifs de responsive design fixés.

5.3 Difficultés rencontrées et solutions apportées

5.3.1 Gestion des conflits de réservation

L'une des premières difficultés rencontrées a été la conception d'un algorithme de détection des conflits de réservation suffisamment précis pour couvrir tous les cas possibles (conflit sur la salle, sur l'enseignant ou sur la classe) tout en restant performant. La solution retenue consiste à effectuer une requête EXISTS sur la table seances avant chaque création, en vérifiant que ni la salle, ni l'enseignant, ni la classe ne sont déjà occupés sur le créneau demandé. Cette vérification

est également réalisée côté client via AJAX pour filtrer les options disponibles dans les listes déroulantes.

5.3.2 Transactions de base de données

La création simultanée d'un compte User et d'un profil Enseignant ou Etudiant a posé la question de l'atomicité : que se passe-t-il si la création du profil échoue après que le User a déjà été créé ? La solution adoptée a été d'encapsuler ces opérations dans une transaction de base de données (DB::beginTransaction(), DB::commit(), DB::rollBack()), garantissant ainsi qu'en cas d'erreur à n'importe quelle étape, aucun enregistrement partiel n'est laissé dans la base.

5.3.3 Gestion des requêtes AJAX

La mise en place de la vérification de disponibilité via AJAX a nécessité la création de routes et de méthodes de contrôleur dédiées retournant des réponses JSON. La difficulté principale résidait dans la synchronisation entre les événements JavaScript côté client (changement de date/horaire) et les appels asynchrones au serveur. L'usage de jQuery et du pattern de callbacks a permis de résoudre cette difficulté.

5.3.4 Gestion des contraintes d'intégrité

La suppression d'entités liées par des clés étrangères (notamment les classes référencées par des étudiants ou des séances) peut déclencher des erreurs MySQL non gérées. La solution adoptée consiste à capturer l'exception Illuminate\Database\QueryException levée par Eloquent lors d'une violation de contrainte d'intégrité référentielle, et à rediriger l'utilisateur avec un message d'erreur explicatif plutôt que de laisser l'application planter.

5.4 Perspectives d'amélioration

Bien qu'UniSalles réponde pleinement aux objectifs fixés dans le cahier des charges, plusieurs axes d'amélioration ont été identifiés pour enrichir la plateforme dans des versions futures :

- **Intégration d'un chatbot** : le cahier des charges prévoit l'intégration d'un assistant conversationnel pour faciliter l'envoi des réclamations et répondre aux questions fréquentes des utilisateurs. Cette fonctionnalité n'a pas été implémentée dans la version actuelle et constitue la principale perspective à court terme.
- **Système de notifications en temps réel** : l'utilisation de WebSockets (via Laravel Echo et Pusher ou Laravel Reverb) permettrait de notifier les utilisateurs en temps réel des modifications importantes (nouvelle séance, réclamation traitée, etc.) sans nécessiter le rechargement de la page.

- **Génération automatique d'emplois du temps** : un algorithme de génération automatique des emplois du temps, prenant en compte les contraintes de disponibilité des salles, des enseignants et des classes, constituerait une évolution majeure de la plateforme.
- **Export PDF des emplois du temps** : la possibilité d'exporter l'emploi du temps au format PDF faciliterait sa consultation hors ligne et son impression.
- **Application mobile** : le développement d'une application mobile native (ou d'une Progressive Web App) améliorerait l'accessibilité de la plateforme pour les utilisateurs sur smartphone.
- **Tableau de bord analytique avancé** : l'ajout de graphiques et de statistiques plus détaillées (taux d'occupation des salles, évolution des réclamations, etc.) enrichirait le tableau de bord du chef de département.
- **Tests automatisés** : la mise en place d'une suite de tests unitaires et d'intégration automatisés avec PHPUnit (intégré à Laravel) améliorerait la fiabilité des déploiements futurs et faciliterait la maintenance du code.

Ce cinquième chapitre clôt la partie technique du rapport. Les tests réalisés confirment la solidité fonctionnelle de la plateforme UniSalles et sa conformité aux exigences du cahier des charges. Les perspectives d'amélioration identifiées tracent la voie d'une évolution continue de l'application.

Conclusion générale

Le projet UniSalles, développé dans le cadre du module de développement web et de génie logiciel de l'Institut des Études Technologiques de Radès, a atteint l'ensemble des objectifs fixés dans le cahier des charges initial. En réponse à une problématique concrète et répandue dans les établissements d'enseignement supérieur, la plateforme offre une solution numérique complète, moderne et sécurisée pour la gestion centralisée des salles universitaires.

Sur le plan fonctionnel, UniSalles propose une palette de fonctionnalités couvrant l'intégralité du cycle de vie d'une réservation de salle : de la planification par le chef de département à la consultation par l'étudiant, en passant par la réservation par l'enseignant et la gestion des réclamations. Le mécanisme de vérification des disponibilités en temps réel via AJAX constitue l'une des innovations techniques les plus significatives de ce projet, offrant une expérience utilisateur fluide et réduisant efficacement les risques de conflits de planification.

Sur le plan technique, ce projet nous a permis d'acquérir une maîtrise approfondie du framework Laravel et de ses composantes principales (Eloquent ORM, Blade, middleware, authentification, sessions), ainsi qu'une expérience pratique des bonnes pratiques de développement web : architecture MVC, gestion des transactions de base de données, sécurité CSRF, gestion des rôles et des accès, et tests fonctionnels.

Sur le plan méthodologique, l'approche Agile Scrum adoptée s'est révélée parfaitement adaptée à ce type de projet. Le découpage en sprints successifs a permis de livrer des fonctionnalités opérationnelles à intervalles réguliers, de recueillir des retours de l'encadrante au fil du développement et d'ajuster les priorités en fonction des besoins identifiés.

Ce projet d'intégration représente pour nous une expérience formatrice exceptionnelle, qui a mobilisé l'ensemble des compétences acquises tout au long de notre cursus universitaire : analyse des besoins, modélisation UML, conception de bases de données, développement back-end et front-end, et validation fonctionnelle. Il constitue également un premier pas concret vers notre future vie professionnelle de développeurs web.

Nous espérons que la plateforme UniSalles, dans sa version actuelle ou dans ses évolutions futures, pourra apporter une contribution tangible à l'amélioration de la gestion administrative et pédagogique de l'IET Radès et, plus largement, servir de modèle à d'autres établissements confrontés aux mêmes défis organisationnels.

Bibliographie et Webographie

Ouvrages et documentation technique

- Otwell, T. (2023). Laravel: The PHP Framework For Web Artisans. Documentation officielle. <https://laravel.com/docs>
- MySQL AB (2023). MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/>
- Bootstrap Team (2023). Bootstrap 5.3 Documentation. <https://getbootstrap.com/docs/5.3/>
- PHP Group (2023). PHP Manual. <https://www.php.net/manual/fr/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Ressources en ligne + outils IA+lien Git :

- Laracasts — Laravel Video Tutorials. <https://laracasts.com>
- Stack Overflow — Communauté de développeurs. <https://stackoverflow.com>
- GitHub — Plateforme de gestion de code source. <https://github.com>
- W3Schools — Tutoriels HTML, CSS, JavaScript. <https://www.w3schools.com>
- MDN Web Docs — Référence JavaScript. <https://developer.mozilla.org>
- phpMyAdmin Documentation. <https://www.phpmyadmin.net/docs/>
- jQuery API Documentation. <https://api.jquery.com>

outils IA : ChatGPT

lien GIT: https://github.com/mdaminehayouni/Gestion_inscription_PL.git

Annexes

Annexe A — Extrait du code source : GestionSalleController

L'extrait ci-dessous illustre la méthode store() du GestionSalleController, responsable de la création d'une nouvelle salle :

```
public function store(Request $request) {
    $request->validate([
        'nomSalle' => 'required|string|max:255',
        'capacite' => 'required|integer|min:1',
        'type' => 'required|string|max:255',
    ]);
    Salle::create($request->only(['nomSalle','capacite','type']));
    return redirect()->route('chef.gestionSalle')
        ->with('success','Salle créée avec succès.');
```

Annexe B — Extrait du code source : Vérification de conflit (Réservation)

L'extrait ci-dessous illustre la logique de détection de conflit lors d'une réservation :

```
$conflict = Seance::where('date', $request->date)
    ->where('heureDebut', $request->heureDebut)
    ->where(function($q) use ($request) {
        $q->where('salle_id', $request->salle_id)
        ->orWhere('classe_id', $request->classe_id)
        ->orWhere('enseignant_id', $request->enseignant_id);
    }->exists());
if ($conflict) {
    return back()->withErrors(['conflit' => 'Conflit horaire détecté.']);
}
```

Annexe C — Diagramme de Gantt du projet

Semaine	Activité	Responsable
S1	Analyse des besoins + Cahier des charges	Équipe
S2	Conception UML + Modèle de base de données	Équipe
S3	Authentification + Gestion des rôles	Équipe
S4	Module Salles + CRUD Chef	Équipe
S5	Module Réservations + Disponibilité AJAX	Équipe
S6	Module Réclamations + Notifications	Équipe
S7	Chatbot (perspective) + Optimisations	Équipe
S8	Tests + Corrections + Rédaction du rapport	Équipe